UNIVERSIDAD POLITÉCNICA DE VICTORIA

IMPLEMENTACIÓN DE FLUENTES DE ESTADO MULTIVALUADOS EN MDP-PROBLOG MEDIANTE DISYUNCIONES ANOTADAS.

T E S I N A

QUE PARA OBTENER EL GRADO DE

INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN

PRESENTA:

ÁNGEL IVÁN CABRERA ROJAS

DIRECTOR

DR. SAID POLANCO MARTAGÓN

CO-DIRECTOR

DR. HÉCTOR HUGO AVILÉS-ARRIAGA

ORGANISMO RECEPTOR

UNIVERSIDAD POLITÉCNICA DE VICTORIA

CIUDAD VICTORIA, TAMAULIPAS, FEBRERO DE 2026

Ciudad Victoria, Tamaulipas, a **25 de Enero de 2026**

UNIVERSIDAD POLITÉCNICA DE VICTORIA
DR. HÉCTOR HUGO AVILÉS ARRIAGA
PRESENTE

La Universidad Politécnica de Victoria tiene a bien presentar a **CABRERA ROJAS ANGEL IVAN** estudiante del programa académico de **INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN**, con número de matrícula **2230343** y seguro facultativo IMSS número **86-18-04-3681-7;** quién deberá realizar su práctica profesional de **ESTADÍA**, a partir del **07 de Enero de 2026** al **17 de Abril de 2026**, con duración de **600 horas** desarrollando el proyecto **"Implementación de disyunciones anotadas en MDP-ProbLog"** que le fué asignado.

Al concluir se le extenderá la carta de liberación al evaluarlo satisfactoriamente y además le solicitaremos su valiosa opinión respondiendo el formulario que se le enviará por mail, referente al desempeño del practicante, la información que nos proporcione, es de vital importancia para la mejora de los programas académicos que ofrece la Universidad Politécnica de Victoria para la formación de profesionistas altamente especializados.

El practicante deberá cumplir con el Reglamento Interno aplicable al personal en su centro de trabajo.

Sin otro particular.

ATENTAMENTE

OTHÓN CANO GARZA
DIRECTOR DE VINCULACIÓN

C.C.P. SAIDPOLANCOMARTAGÓN
ASESOR INSTITUCIONAL

UNIVERSIDAD POLITÉCNICA DE VICTORIA

Victoria Tamaulipas, a 25 de Enero del 2026
Asunto: Carta de Aceptación

M.A OTHÓN CANO GARZA
DIRECTOR DE VINCULACIÓN
UNIVERSIDAD POLITÉCNICA DE VICTORIA
PRESENTE

Hacemos de su conocimiento que hemos aceptado al estudiante **CABRERA ROJAS ANGEL IVAN** del programa académico de **INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN**, con número de matrícula **2230343** de la Universidad Politécnica de Victoria, para realizar su **ESTADÍA** en nuestra empresa **UNIVERSIDAD POLITÉCNICA DE VICTORIA**, durante el periodo comprendido del día **07 de Enero de 2026** al **17 de Abril de 2026**, el estudiante estará colaborando en el proyecto **"Implementación de disyunciones anotadas en MDP-ProbLog"** con una carga horaria total de **600 horas**

Al concluir satisfactoriamente sus prácticas profesionales, se le entregará al estudiante su carta de liberación debidamente formalizada.

Con el objetivo de colaborar en la mejora de los programas académicos de la Universidad Politécnica de Victoria, estamos de acuerdo en responder a la brevedad posible el formulario de evaluación del desempeño del practicante previo a emitir la liberación de **ESTADÍA**.

Sin otro particular.

ATENTAMENTE
DR. HÉCTOR HUGO AVILÉS ARRIAGA
ASESOR EMPRESARIAL

C.C.P. ARCHIVO

Ing. Juan Camaney de Alba Rojas
PRESENTE

INSERTAR AQUÍ
DOCUMENDO
DE CARTA
DE LIBERACION
DEBIDAMENTE
FIRMADO

CARTA DE ACEPTACIÓN DEL DOCUMENTO PARA SU IMPRESIÓN

Cd. Victoria, Tamaulipas a x

Ángel Iván Cabrera Rojas
PRESENTE

Le comunico que el Programa Académico de Ingeniería en Tecnologías de la Información le ha otorgado la autorización para la impresión de su Tesina de Estadía Práctica cuyo título es:

Implementación de fluentes de estado multivaluados en MDP-ProbLog mediante disyunciones anotadas.

ATENTAMENTE

Dr. Said Polanco Martagón
ASESOR INSTITUCIONAL

c.c.p Director de programa académico

EVALUACIÓN DE ESTADÍA
Rúbrica para evaluación de la presentación y el reporte de estadía

Nombre del alumno: **ÁNGEL IVÁN CABRERA ROJAS** Calificación final: _____

Periodo: **X**

Ponderación	Aspecto a Evaluar	Competente 10	Independiente 9	Básico Avanzado 8	No Competente 5
40	Resultados y Actividades	Estrechamente relacionados al perfil de egreso de su programa académico	Parcialmente relacionados al perfil de egreso de su programa académico	Escasamente relacionados al perfil de egreso de su programa académico	Escasamente relacionados al perfil de egreso de su programa académico
30	Exposición de las actividades de la estadía	Detalladas y sustentadas con respecto a los resultados que se obtuvieron	Detalladas y sustentadas parcialmente con respecto a los resultados que se obtuvieron	Detalladas parcialmente con respecto a los resultados que se obtuvieron	Detalladas escazamente con respecto a los resultados que se obtuvieron
10	Material visual Lenguaje verbal	Uso el lenguaje y la terminología apropiadas; El material visual está organizado, adecuado y suficiente	Uso el lenguaje y la terminología apropiadas El material visual está parcialmente organizado y es suficiente	Uso el lenguaje y la terminología son parcialmente apropiadas; El material visual está parcialmente organizado y es suficiente	Uso el lenguaje y terminología es inapropiado; El material visual no está organizado y es insuficiente
10	Exposición en Idioma Inglés	Pronunciation is clear so language is easily understood (2.5) Uses fluent connected speech, occasionally disrupted by search for correct form of expression (2.5) Uses topic related vocabulary without problems (2.5) Responds to questions using varied and descriptive vocabulary and language structures (2.5)	Pronunciation is understandable, but there are slight errors (2.25) Speech is connected but frequently disrupted by search for correct form of expression (2.25) Uses some topic related vocabulary sufficient to communicate ideas (2.25) Responds to questions using simple but accurate vocabulary and language structures (2.25	Pronunciation is understandable most of the time, marked native accent and many errors (2) Speaks with simple sentences, sometimes not connected, but is understood (2) Uses basic vocabulary to communicate ideas (2) Partly responds to simple questions, with limited vocabulary and language structures (2)	Pronunciation makes language very difficult to understand (1) Uses one-word/two-word utterances (1) Unable to communicate ideas due to lack of vocabulary (1) Uses isolated words or sentence fragments to respond to questions (1)
5	Respuesta a los cuestiona-mientos de los evaluadores	Clara y satisfactoria	Clara y parcialmente satisfactoria	Clara e insuficiente	Confusa e insuficiente
5	Autorización de tesina en tiempo y forma	Presenta en tiempo y forma	Presenta en tiempo y forma con la mayoría de requerimientos solicitados	Presenta en tiempo y con algunas limitantes de los requerimientos solicitados.	Presenta fuera de tiempo y con los mínimos requerimientos solicitados.

Dr. Said Polanco Martagón	X	
ASESOR INSTITUCIONAL	EVALUADOR	EVALUADOR DE INGLÉS

REGISTRO DE EVALUACIÓN DE EXPOSICIÓN DE ESTADÍA

Siendo las X horas del día X, el alumno **Ángel Iván Cabrera Rojas**, del programa académico **Ingeniería en Tecnologías de la Información**, con matricula **2230343**, presentó la exposición de la estadía realizada durante el cuatrimestre **x**, en la **Universidad Politécnica de Victoria**, con el proyecto titulado **Implementación de fluentes de estado multivaluados en MDP-ProbLog mediante disyunciones anotadas.**.

Una vez concluido el proceso de evaluación, y con base a la rúbrica establecida para éste propósito, se determina que la calificación de la estadía es _____.

_____
Dr. Said Polanco Martagón
ASESOR INSTITUCIONAL

_____
X
EVALUADOR

_____

EVALUADOR DE INGLÉS

Agradecimientos

Resumen

La Programación Lógica Probabilística (PLP) es un paradigma que combina la capacidad de representación de la lógica de primer orden con la teoría de la probabilidad para el manejo de la incertidumbre. Este paradigma permite modelar una amplia variedad de dominios que abarcan situaciones de la vida real en donde existen interacciones complejas entre conjuntos de elementos y distintos niveles de incertidumbre. Dentro de este dominio, el framework MDP-ProbLog utiliza las capacidades del lenguaje ProbLog para la especificación y resolución de Procesos de Decisión de Markov (MDPs) de horizonte infinito utilizando una representación factorizada, lógica y relacional.

Sin embargo, la arquitectura base de este sistema presenta una limitación estructural ya que la representación de los fluentes de estado se restringe exclusivamente a variables de Bernoulli independientes (booleanas). Debido a esto, para describir variables que por naturaleza son multivaluadas, el usuario se ve obligado a utilizar técnicas para descomponerlas en múltiples predicados binarios. En el caso de MDP-ProbLog, esta restricción resulta en un proceso de modelado tedioso, poco práctico y propenso a inconsistencias semánticas.

Se reporta el desarrollo de una extensión al núcleo del framework MDP-ProbLog para solucionar esta problemática. El trabajo abarca la revisión de la arquitectura del código fuente, el planteamiento y diseño de la extensión, la programación, la ejecución de pruebas y su respectiva documentación. Mediante modificaciones a los módulos de gestión de estados y del motor de inferencia, se integró el soporte para fluentes de estado multivaluados utilizando el mecanismo nativo de las Disyunciones Anotadas (Annotated Disjunctions). Como resultado de este desarrollo, el modelado de MDPs que requieren variables con dominios multivaluados se convierte en un proceso semánticamente íntegro, mucho más directo y natural para el usuario.

Palabras clave: Programación Lógica Probabilística, MDP-ProbLog, Disyunciones Anotadas, Fluentes Multivaluados, Espacio de Estados.

Summary

Probabilistic Logic Programming (PLP) is a paradigm that combines the representational power of first-order logic with probability theory to handle uncertainty. This paradigm allows for the modeling of a wide variety of domains encompassing real-world situations where complex interactions exist among sets of heterogeneous elements and varying levels of uncertainty. Within this context, the MDP-ProbLog framework stands out, leveraging the capabilities of the ProbLog language for the specification and resolution of infinite-horizon Markov Decision Processes (MDPs) using a factored, logical, and relational representation.

However, the base architecture of this system presents a structural limitation: the representation of state fluents is restricted exclusively to independent Bernoulli (Boolean) variables. Due to this, to model variables that are naturally multi-valued, the user is forced to use techniques to decompose them into multiple binary predicates. In the case of MDP-ProbLog, this restriction results in a modeling process that is tedious, prone to semantic inconsistencies, and unnatural.

This thesis reports the development of an extension to the core of the MDP-ProbLog framework to address this problem. The work encompasses the review of the source code architecture, the planning and design of the extension, programming, testing, and its respective documentation. Through modifications to the state management modules and the inference engine, support for multi-valued state fluents was integrated using the native mechanism of Annotated Disjunctions. As a result of this development, the modeling of MDPs requiring variables with multi-valued domains becomes a semantically sound, much more direct, and natural process for the user.

Keywords: Probabilistic Logic Programming, MDP-ProbLog, Annotated Disjunctions, Multi-valued Fluents, State Space.

Índice

Agradecimientos vii

Resumen viii

Summary ix

Índice x

1. Introducción 1
- 1.1. Antecedentes . . . 2
- 1.2. Definición del Problema . . . 4
- 1.3. Objetivos . . . 5
  - 1.3.1. Objetivo General . . . 5
  - 1.3.2. Objetivos Específicos . . . 5
- 1.4. Justificación . . . 6
- 1.5. Alcances y Limitaciones . . . 7
  - 1.5.1. Alcances . . . 7
  - 1.5.2. Limitaciones . . . 8

2. Estado del Arte 9
- 2.1. MDPs Factorizados y el Problema de la Representación de Estados . . . 9
- 2.2. Programación Lógica Probabilística para la Resolución de MDPs . . . 10
- 2.3. Disyunciones Anotadas como mecanismo de exclusión mutua . . . 11
- 2.4. Compilación de conocimiento para inferencia eficiente . . . 11
- 2.5. Análisis Comparativo . . . 12
- 2.6. Conclusión del Estado del Arte . . . 13

3. Marco Teórico 14
- 3.1. Grid Estocástico . . . 14
- 3.2. Programación Lógica . . . 15
- 3.3. Programación Lógica Probabilística . . . 16
  - 3.3.1. Semántica de distribución . . . 16
- 3.4. Lenguaje ProbLog . . . 18
- 3.5. Disyunciones Anotadas . . . 19
- 3.6. Conteo de Modelos Ponderados y Compilación de Conocimiento . . . 21
- 3.7. Procesos de Decisión de Markov . . . 22
- 3.8. MDPs Factorizados y Fluentes de Estado . . . 24
- 3.9. El framework MDP-ProbLog . . . 25

4. Diseño e Implementación 27
- 4.1. Análisis de la arquitectura original . . . 27
  - 4.1.1. Componentes del sistema original . . . 27
  - 4.1.2. Flujo de ejecución . . . 28
  - 4.1.3. Limitaciones en la gestión de fluentes . . . 28

4.1.4. Identificación de puntos de extensión . . . 29
4.2. Diseño de la extensión . . . 29
4.2.1. FluentSchema: estructura factorizada del espacio de estados . . . 29
4.2.2. FluentClassifier: mecanismo de clasificación automática . . . 30
4.2.3. Codificación *mixed-radix* del espacio de estados . . . 31
4.2.4. Sintaxis de usuario para fluentes multivaluados . . . 32
4.3. Implementación . . . 34
4.3.1. Adaptador al motor de inferencia . . . 34
4.3.2. Coordinador central del MDP . . . 34
4.3.3. Adaptaciones al algoritmo de resolución . . . 35
4.3.4. Herramientas de verificación y exportación . . . 36
4.4. Retrocompatibilidad . . . 36

5. Experimentación y Resultados **38**
5.1. Diseño experimental . . . 38
5.1.1. Dominio de prueba . . . 38
5.1.2. Codificaciones comparadas . . . 39
5.1.3. Métricas de evaluación . . . 41
5.1.4. Protocolo de medición . . . 41
5.1.5. Entorno de pruebas . . . 42
5.2. Verificación de corrección funcional . . . 42
5.3. Simplificación del proceso de modelado . . . 43
5.4. Impacto computacional . . . 44
5.4.1. Tamaño del circuito compilado . . . 44
5.4.2. Tiempos de ejecución . . . 44
5.5. Discusión . . . 45
5.5.1. Corrección y convergencia . . . 45
5.5.2. Simplificación del modelado . . . 45
5.5.3. Comportamiento computacional . . . 46
5.5.4. Relación con las predicciones del Marco Teórico . . . 46
5.5.5. Limitaciones de la evaluación . . . 46

Índice de figuras **47**

Índice de cuadros **48**

Índice de algoritmos **49**

Referencias **50**

1. Introducción

Dentro del ámbito de la Inteligencia Artificial, la Programación Lógica Probabilística se ha consolidado como un paradigma expresivo y formalmente fundamentado para la representación del conocimiento bajo incertidumbre, al combinar la capacidad deductiva de la programación lógica con el tratamiento de la probabilidad [1]. Uno de los desarrollos más representativos de este paradigma es *ProbLog*, un lenguaje que extiende Prolog mediante la introducción de hechos probabilísticos y cuya semántica se sustenta en una distribución sobre mundos posibles [2]. ProbLog permite calcular la probabilidad de que una consulta lógica sea verdadera dado un conjunto de evidencias, transformando la inferencia probabilística en un problema de evaluación eficiente sobre estructuras compiladas [3].

Asimismo, en el campo de la toma de decisiones secuenciales, los Procesos de Decisión de Markov (MDPs) se han establecido como el estándar matemático para modelar agentes que actúan en entornos estocásticos. Formalizados a mediados del siglo XX con los trabajos de Richard Bellman sobre programación dinámica [4], los MDPs proporcionan un marco formal para optimizar una función de utilidad a lo largo del tiempo. En su formulación tabular clásica, cada estado del sistema se trata como una entidad indivisible y las transiciones se almacenan en una tabla que lista explícitamente la probabilidad de pasar de cada estado a cada otro estado bajo cada acción posible. A medida que aumenta el número de variables que describen el estado del sistema, el tamaño de dicha tabla crece de manera exponencial, un fenómeno que Bellman denominó la "maldición de la dimensionalidad" [4]. Esta particularidad, hace que tanto la especificación como la resolución de MDPs se vuelvan computacionalmente costosas conforme crece la complejidad del dominio [5].

Integrando la expresividad lógica de ProbLog con la capacidad de decisión de los MDPs, Bueno et al. propusieron el framework **MDP-ProbLog** [6]. Esta herramienta codifica estados, acciones y recompensas dentro de la sintaxis de la programación lógica, permitiendo representar MDPs de horizonte infinito, es decir, problemas de decisión secuencial sin un punto de terminación predefinido donde el agente busca una estrategia óptima a largo plazo, de manera relacional y compacta. En lugar de tratar cada estado como una entidad indivisible, MDP-ProbLog adopta una representación factorizada en la que se descompone el estado global en un conjunto de variables independientes denominadas *fluentes*, de modo que la dinámica del sistema se describe en términos de cómo cambia cada variable individual. Esta representación permite *especificar* el MDP de forma compacta y legible, aunque la resolución del problema mediante el algoritmo de iteración de valor continúa siendo enumerativa y, por tanto, sujeta a la explosión combinatoria del espacio de estados.

1.1. Antecedentes

La idea de utilizar lenguajes declarativos para especificar MDPs no surge con MDP-ProbLog. Boutilier, Dean y Hanks [7] propusieron a finales de la década de 1990 el uso de representaciones factorizadas para abordar la maldición de la dimensionalidad, describiendo las transiciones de estado mediante redes bayesianas dinámicas en lugar de tablas monolíticas. Posteriormente, herramientas como SPUDD [8] explotaron diagramas de decisión algebraicos para resolver MDPs factorizados de manera simbólica.

En el contexto de la Programación Lógica Probabilística, Nitti, Belle y De Raedt [9] demostraron que los lenguajes probabilísticos basados en lógica podían servir como especificación declarativa de problemas de planificació mediante técnicas de muestreo para dominios tanto discretos como continuos. MDP-ProbLog [6] se inscribe en esta línea al proponer una codificación completamente declarativa de MDPs de horizonte infinito dentro de la sintaxis de ProbLog, sustituyendo el muestreo por inferencia exacta mediante Conteo de Modelos Ponderados.

MDP-ProbLog ha sido adoptado como base tecnológica por el equipo de investigación liderado por el Dr. Héctor Hugo Avilés-Arriaga en la Universidad Politécnica de Victoria, el cual lo ha empleado en el desarrollo de sistemas de navegación y toma de decisiones para vehículos autónomos [10]. Sin embargo, al tratarse de un framework que únicamente admite fluentes booleanos, la construcción de los modelos requiere adaptar la descripción del dominio a las limitaciones de la herramienta, en lugar de permitir su representación en términos naturales. Esto en sí mismo es una limitación estructural inherente del framework, ya que todos los fluentes de estado se definen exclusivamente como variables de Bernoulli independientes.

Para ilustrar el problema de manera concreta, considérese un dominio de navegación en cuadrícula, un escenario canónico en la literatura de MDPs [11, 12] en el cual, un agente debe desplazarse desde una posición inicial hasta una celda meta, seleccionando en cada paso una de cuatro acciones de movimiento. La posición del agente compone una partde del estado del sistema y puede describirse mediante dos variables: la fila y la columna que ocupa.

Debido a que cada fluente de estado solo admite dos valores (verdadero o falso), la posición de un agente no puede codificarse directamente como un par de coordenadas, ya que cada coordenada naturalmente toma más de dos valores. Por ello, la representación booleana de MDP-ProbLog resulta insuficiente.

En el caso del grid de 2×3, donde existen 6 celdas posibles, se requieren al menos 3 fluentes booleanos para distinguir 6 estados distintos utilizando únicamente variables de dos valores, ya que 2 variables solo permiten $2^2 = 4$ combinaciones, insuficientes para representar las 6 celdas. Con 3 variables booleanas se obtienen $2^3 = 8$ combinaciones, cantidad suficiente para asignar una combinación única a cada celda.

Como se observa en la Tabla 1, las 8 combinaciones que generan los 3 fluentes booleanos exceden las 6 celdas reales del grid. A lo largo del documento, se denominará *estados espurios* a las combinaciones sobrantes, es decir, configuraciones que el sistema de inferencia considera válidas pero que carecen de correspondencia física en el dominio.

Tabla 1: Codificación binaria de las 6 celdas de un grid de 2×3.

b_1	b_2	b_3	**Celda asignada**
0	0	0	Celda (1,1)
0	0	1	Celda (1,2)
0	1	0	Celda (1,3)
0	1	1	(sin correspondencia)
1	0	0	Celda (2,1)
1	0	1	Celda (2,2)
1	1	0	Celda (2,3)
1	1	1	(sin correspondencia)

La Figura 1 contrasta ambas representaciones del mismo grid: a la izquierda, la descripción mediante las variables de fila (x) y columna (y), que genera exactamente 6 estados; a la derecha, la codificación booleana mediante tres fluentes binarios (b_1, b_2, b_3), que produce 8 combinaciones de las cuales 2 son espurias.

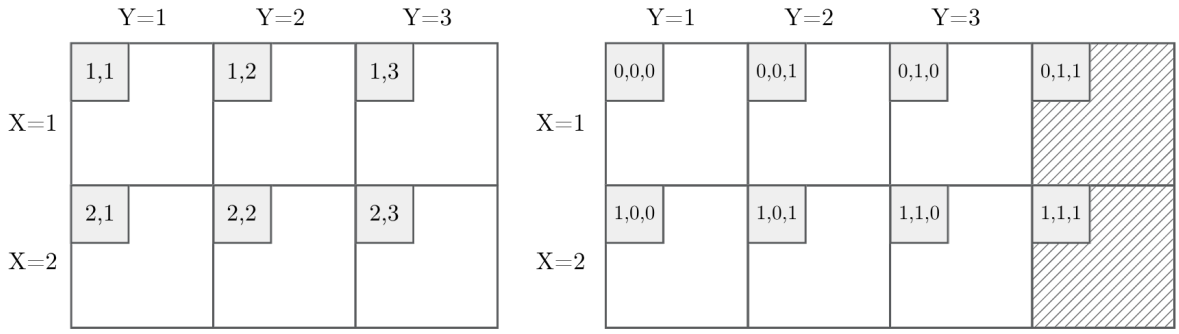

Figura 1: Representación por coordenadas vs. codificación booleana con estados inalcanzables para el grid 2×3.

Este procedimiento de codificación sigue un patrón general. Si se desea representar k estados mutuamente excluyentes se requieren al menos $\lceil \log_2(k) \rceil$ fluentes booleanos, los cuales generan $2^{\lceil \log_2(k) \rceil}$ combinaciones. Siempre que k no sea una potencia exacta de 2, el espacio de estados contendrá combinaciones sin significado claro y que el modelador debe gestionar manualmente. La Tabla 2 ilustra cómo este desajuste crece con el tamaño del dominio.

Tabla 2: Escalamiento de estados espurios en la codificación binaria por tamaño de grid.

Grid	**Celdas** (k)	**Bits necesarios**	**Combinaciones** (2^{bits})	**Espurios**
2×3	6	3	8	2
3×3	9	4	16	7
4×4	16	4	16	0
5×5	25	5	32	7
6×6	36	6	64	28

En un grid de 6×6, por ejemplo, 28 de las 64 combinaciones binarias son espurias, ejemplificando de manera clara las problematicas inherentes a los espacios de estados con variables categóricas mediante la representación que el framework impone actualmente.

1.2. Definición del Problema

Como se ilustró en la sección anterior con el grid de navegación, MDP-ProbLog carece de un mecanismo nativo para representar variables de estado que posean más de dos valores mutuamente excluyentes. Cada fluente se trata como una variable de Bernoulli independiente, lo que obliga a representar cualquier variable multivaluada mediante un conjunto de predicados booleanos cuyas combinaciones codifiquen los valores posibles.

Las consecuencias de esta restricción se extienden a cualquier dominio que requiera variables categóricas. Ejemplos típicos de esto, pueden ser, el estado de un semáforo, el carril que ocupa un vehículo o la fase de un proceso; todas ellas variables que, por su naturaleza, se describen mediante más de dos estados posibles.

En todos estos casos, para representar correctamente este tipo de variables, el usuario debe tener en cuenta estas limitaciones, mientras que el solver puede llegar a ejecutar la iteración de valor sobre un espacio de estados inflado con configuraciones inalcanzables, invirtiendo cómputo en estados que no contribuyen a la política resultante.

Para dimensionar cómo esta limitación afecta el modelado, es útil observar cómo herramientas históricas lidiaron con la misma barrera. La Tabla 3 muestra un ejemplo clásico en la literatura de MDPs factorizados: el dominio de planificación de procesos de manufactura (*Factory*), introducido para evaluar el algoritmo SPUDD [8]. En este escenario se requieren variables categóricas ternarias para modelar con precisión el entorno, tal como la calidad de la pintura de una pieza (buena, deficiente o inexistente).

Tabla 3: Codificación binaria forzada para una variable ternaria en el dominio *Factory*.

Bit 1	**Bit 2**	**Calidad de la pintura**
0	0	Buena (*good*)
0	1	Deficiente (*poor*)
1	0	Inexistente (*false*)
1	1	Estado espurio (*unreachable state*)

Al igual que ocurría en SPUDD, la codificación binaria en MDP-ProbLog introduce inevitablemente estados inalcanzables, lo que provoca que por un lado, wzista un aumento en la complejidad del modelado, debido a que el usuario no puede simplemente declarar los posibles valores de una variable. Esta limitación se traduce en una ineficiencia computacional.

En síntesis, la representación exclusivamente booleana produce modelos que son simultáneamente más costosos de escribir para el humano y menos eficientes de resolver para la máquina, forzando al dominio a adaptarse a las limitaciones de la herramienta.

1.3. Objetivos

1.3.1. Objetivo General

Diseñar e integrar un mecanismo de representación de fluentes de estado multivaluados en el núcleo del framework MDP-ProbLog utilizando Disyunciones Anotadas (Annotated Disjunctions), con el fin de proveer una sintaxis de modelado más natural, asegurar la consistencia en los procesos de decisión estocásticos y evaluar el impacto computacional derivado de esta nueva expresividad.

1.3.2. Objetivos Específicos

- Analizar la arquitectura interna actual de MDP-ProbLog, identificando los módulos críticos encargados de la gestión del espacio de estados y la inferencia probabilística para determinar los puntos de extensión.
- Desarrollar una estructura de datos y adaptar la lógica de instanciación (*grounding*) que permita procesar fluentes multivaluados frente a los fluentes binarios tradicionales.
- Implementar el soporte para Disyunciones Anotadas en la definición de las transiciones de estado, garantizando la exclusión mutua nativa de los valores de una variable.
- Desarrollar las herramientas de inspección que permitan examinar las matrices de transición, las tablas de recompensas y el historial de convergencia de los modelos resueltos.

1.4. Justificación

La extensión propuesta representa un paso necesario para la evolución del framework debido a las limitaciones impuestas por el modelado estrictamente binario. En ausencia de variables categóricas nativas, la arquitectura original transfiere al modelador la tarea de restringir matemáticamente el entorno. Es por ello que usuario debe diseñar y escribir reglas de integridad manuales para evitar que el sistema asigne valores contradictorios a una misma entidad (por ejemplo, evaluar a un agente que ocupa dos posiciones simultáneamente). Esta práctica introduce ruido en el código, oscureciendo la lógica real del proceso de decisión bajo capas de restricciones artificiales.

Se soluciona el problema al trasladar la carga de validación del humano hacia el motor lógico mediante las Disyunciones Anotadas, las cuales son un constructo nativo de la Programación Lógica Probabilística [13] que garantiza la exclusión mutua entre los valores de una variable por construcción semántica. Al delegar esta responsabilidad a la estructura interna de estas disyunciones, el código ProbLog resultante recobra legibilidad y el modelador concentra su esfuerzo en definir la dinámica y las recompensas del entorno, reduciendo la carga cognitiva, acortando los tiempos de desarrollo y eliminando una fuente recurrente de errores de diseño.

Además de las mejoras en la usabilidad, la precisión de esta nueva representación resuelve parte de las deficiencias en el rendimiento computacional. Durante el proceso de resolución original, el algoritmo de iteración de valor desperdicia ciclos de cómputo evaluando configuraciones inalcanzables, invirtiendo recursos en estados que jamás contribuirán a la política resultante. Al garantizar que el espacio de estados contenga únicamente los escenarios que el dominio contempla, el sistema elimina las configuraciones redundantes introducidas por la codificación booleana. Este ahorro computacional es directamente proporcional al número de estados espurios evitados; beneficio que se acumula considerablemente en dominios que utilizan múltiples variables categóricas.

Finalmente, la integración de las Disyunciones Anotadas para modelar fluentes multivaluados no requiere de la modificación de los algoritmos de compilación ni de razonamiento probabilístico. La propuesta extiende la expresividad de la herramienta sin comprometer su arquitectura subyacente ni su retrocompatibilidad con los modelos booleanos existentes.

1.5. Alcances y Limitaciones

1.5.1. Alcances

El proyecto abarca el diseño, la implementación y la validación de una extensión al núcleo del framework MDP-ProbLog que incorpora soporte para fluentes de estado multivaluados. El desarrollo comprende los siguientes elementos:

1. **Representación factorizada del espacio de estados.** El rediseño de las estructuras de datos necesarias para que el espacio de estados admita factores con más de dos valores posibles, preservando la capacidad de iteración y la indexación eficiente sobre el conjunto completo de estados.

2. **Clasificación automática de fluentes.** Se implementó un mecanismo que determina automáticamente si un fluente declarado por el usuario es booleano o multivaluado, a partir del análisis de las Disyunciones Anotadas presentes en el programa. El usuario también puede especificar el tipo de forma explícita.

3. **Integración con el motor de inferencia.** Se adaptó la interfaz entre MDP-ProbLog y el motor de inferencia de ProbLog para soportar la inyección y el manejo de Disyunciones Anotadas como representación nativa de los fluentes multivaluados.

4. **Adaptación del algoritmo de resolución.** Se reestructuró el flujo de evaluación de transiciones y el algoritmo de Iteración de Valor para operar sobre transiciones agrupadas por factor, de modo que cada variable de estado se evalúa de acuerdo con su cardinalidad real.

5. **Herramientas de inspección.** Se desarrollaron utilidades para exportar las matrices de transición, las tablas de recompensas y el historial de convergencia, facilitando la verificación y el análisis de los modelos resueltos.

6. **Retrocompatibilidad.** La extensión preserva el funcionamiento de los modelos booleanos existentes sin requerir modificaciones por parte del usuario.

La validación funcional se realiza sobre dominios basados en grids, específicamente el grid de Mitchell que constituye un problema canónico en la literatura de MDPs con soluciones óptimas conocidas. En cada dominio se contrastan dos formulaciones del mismo problema, una con codificación booleana y otra con fluentes multivaluados, evaluando la simplificación del proceso de modelado, la corrección de la política resultante y el comportamiento del solver en términos de convergencia y tamaño del espacio de estados.

1.5.2. Limitaciones

El desarrollo se restringe a MDPs de horizonte infinito con espacios de estados discretos y completamente observables. Los siguiente puntos delimitan el alcance técnico de la extensión:

1. **Enumeración explícita del espacio de estados.** El alcance de esta extensión se limita a la capa de representación del espacio de estados; la estrategia de resolución enumerativa del framework original permanece sin modificación. El algoritmo de Iteración de Valor continúa recorriendo todos los pares estado-acción en cada iteración, con un costo proporcional al número total de estados por el número de acciones. Dominios con múltiples variables de estado o con variables de cardinalidad alta producen espacios de estados cuya enumeración completa se vuelve computacionalmente costosa.

2. **Capa de representación exclusivamente.** El trabajo no modifica los algoritmos de inferencia exacta ni los compiladores de circuitos lógicos utilizados internamente por ProbLog. El efecto de las Disyunciones Anotadas sobre el tamaño de los circuitos compilados es un resultado emergente del compilador subyacente, no una optimización introducida por esta extensión.

3. **Convención de declaración de fluentes.** Todos los valores posibles de un fluente multivaluado deben declararse bajo el mismo nombre de predicado. Esta convención permite que el sistema identifique automáticamente los grupos mutuamente excluyentes, pero impide representar un fluente cuyos valores se distribuyan entre predicados distintos.

4. **Independencia entre variables de estado.** Las reglas de transición del programa deben respetar independencia entre las variables de estado. El sistema evalúa la probabilidad de transición de cada variable por separado y combina los resultados asumiendo que los factores son estocásticamente independientes. Si las reglas del usuario introducen dependencias entre variables que violan este supuesto, el sistema no lo detecta ni emite advertencias.

5. **Dominios de evaluación.** Las pruebas se ejecutan sobre dominios de grid discretos de dimensión reducida, seleccionados por la disponibilidad de soluciones de referencia en la literatura. Los resultados no se extrapolan a dominios de gran escala ni a entornos físicos en tiempo real.

Una vez delimitado el problema, los objetivos y los alcances de este trabajo, el siguiente capítulo analiza el estado del arte y trabajos que sustentan la propuesta, revisando los enfoques existentes para la representación factorizada de MDPs y el papel de la Programación Lógica Probabilística en la toma de decisiones secuenciales.

2. Estado del Arte

El presente capítulo rastrea la evolución de las tres líneas de investigación cuya intersección define el problema que esta tesina aborda: (1) la representación factorizada de los MDPs y el problema de las variables de estado; (2) el uso de la Programación Lógica Probabilística (PLP) como lenguaje de especificación para la toma de decisiones secuenciales; y (3) los mecanismos de exclusión mutua y compilación de conocimiento para la inferencia eficiente. El recorrido muestra cómo cada uno de ellos ha aportado los elementos necesarios para fundamentar la extensión propuesta.

2.1. MDPs Factorizados y el Problema de la Representación de Estados

A medida que los dominios de planificación crecían en complejidad, quedó claro que ningún método que operara sobre estados individuales podría escalar ya que el tamaño del espacio de estados crece exponencialmente con el número de variables, y los algoritmos clásicos de programación dinámica requieren iterar sobre cada estado [5]. Este crecimiento fue reconocido por la como el obstáculo central que debía resolverse para hacer los MDPs aplicables a problemas reales, y motivó una línea de investigación dedicada a explotar la estructura interna del dominio para eludir la enumeración explícita.

Con este objetivo, Boutilier, Dearden y Goldszmidt propusieron la representación factorizada de MDPs, donde las transiciones se codifican mediante Redes Bayesianas Dinámicas (DBNs) y las funciones de valor se manipulan como árboles de decisión [14]. En esta representación, cada variable de estado posee su propia tabla de probabilidad condicional, y los algoritmos de programación dinámica operan sobre agregaciones de estados en lugar de iterar sobre cada estado individual. Su algoritmo *Structured Policy Iteration* (SPI) demostró que era posible resolver MDPs sin enumerar explícitamente el espacio de estados, siempre que la estructura del problema exhibiera independencias condicionales aprovechables.

En la misma línea, Hoey *et al.* desarrollaron SPUDD, un algoritmo de iteración de valor que reemplaza los árboles de decisión por Diagramas de Decisión Algebraicos (ADDs) [8]. Los ADDs, al ser grafos dirigidos acíclicos, permiten compartir subgrafos isomorfos, logrando reducciones de hasta 30 veces en el número de nodos respecto a las representaciones arbóreas. SPUDD demostró la resolución exacta de MDPs con hasta 63 millones de estados.

Tanto SPI como SPUDD operan exclusivamente sobre variables booleanas debido a la descomposición de Shannon que subyace a ambas estructuras. Cada nodo interno ramifica sobre una variable binaria [8]. Cuando el dominio contiene variables multivaluadas, el modelador debe descomponerlas en múltiples variables binarias. Hoey *et al.* reconocen esta limitación al señalar que una variable ternaria debe expandirse en dos binarias, lo que aumenta el espacio de estados por un factor de 4/3 por variable e introduce estados semánticamente imposibles [8]. Esta binarización forzada constituye precisamente la limitación que se busca corregir en MDP-ProbLog.

En el ámbito de los lenguajes de descripción para planificación, RDDL (*Relational Dynamic Influence Diagram Language*) [15] sí contempla fluentes multivaluados de manera nativa, incluyendo variables enumeradas, enteras y continuas. No obstante, RDDL adopta un DBN estratificado por niveles como representación semántica, donde cada variable de un paso temporal sólo puede depender de variables de niveles anteriores [15]. Esta estructuración acíclica impide modelar dependencias cíclicas entre predicados aterrizados, como las que surgen en dominios con relaciones simétricas o transitivas; MDP-ProbLog resuelve este problema al heredar de Prolog la resolución SLD, capaz de manejar definiciones recursivas sobre el programa aterrizado [6].

2.2. Programación Lógica Probabilística para la Resolución de MDPs

La comunidad de MDPs factorizados desarrolló representaciones cada vez más eficientes del espacio de estados. Años más tarde, la Programación Lógica Probabilística propuso lenguajes relacionales capaces de razonar bajo incertidumbre [16]. La pregunta que eventualmente conectó ambas líneas de investigación fue si los motores de inferencia exacta de ProbLog podían adaptarse para resolver, en lugar de consultas estáticas, problemas de decisión secuencial en los que el agente debe optimizar su comportamiento a lo largo del tiempo.

La extensión de ProbLog hacia la toma de decisiones se materializó primero en DTProbLog [17]. Este sistema añadió al lenguaje la capacidad de modelar problemas de decisión episódicos mediante variables de decisión y atributos de utilidad, permitiendo encontrar la estrategia que maximiza la utilidad esperada. Sin embargo, DTProbLog está diseñado exclusivamente para problemas de un solo paso y no soporta la resolución de MDPs de horizonte infinito, donde es necesario optimizar una política completa a lo largo del tiempo. De esta forma, estableció la prueba de concepto de que ProbLog era extensible hacia la optimización bajo incertidumbre, aunque la resolución de MDPs requería un mecanismo adicional para propagar valor a través de horizontes de tiempo arbitrarios.

Como derivado de este trabajo, Bueno *et al.* propusieron MDP-ProbLog, que extiende la sintaxis de ProbLog con predicados reservados para declarar fluentes de estado, acciones y utilidades, y emplea programas de distribución de transición de dos pasos temporales para representar la dinámica del MDP [6]. El solver implementa iteración de valor sobre el circuito compilado mediante Weighted Model Counting (WMC), demostrando formalmente la equivalencia entre ejecutar el programa probabilístico y realizar un backup de Bellman. Esta equivalencia garantiza que la evaluación sobre el circuito produce exactamente las probabilidades de transición requeridas, permitiendo que el solver converja a políticas óptimas sin enumerar el espacio de estados. Una ventaja distintiva frente a lenguajes como RDDL es su capacidad para representar dependencias cíclicas entre predicados aterrizados gracias a la resolución SLD de Prolog [6].

2.3. Disyunciones Anotadas como mecanismo de exclusión mutua

En paralelo al desarrollo de los MDPs factorizados y de MDP-ProbLog, una línea independiente de investigación dentro de la Programación Lógica Probabilística se ocupaba de un problema distinto: cómo representar eventos con múltiples resultados mutuamente excluyentes sin imponer restricciones externas al programa. Vennekens, Verbaeten y Bruynooghe respondieron a este problema con los *Logic Programs with Annotated Disjunctions* (LPADs) [13], demostrando que es posible codificar la exclusión mutua de manera nativa en la cabeza de las reglas lógicas. Esta propiedad elimina la necesidad de restricciones manuales y evita la generación de estados semánticamente imposibles, dos problemas que los enfoques booleanos independientes trasladan al modelador.

En trabajo posterior, Vennekens, Denecker y Bruynooghe formalizaron una interpretación causal de las ADs mediante *CP-Logic* [18], demostrando que el cuerpo de cada regla representa una causa y los disyuntos en la cabeza sus posibles efectos, cada uno con una probabilidad de ocurrencia. Este alineamiento es directo con la dinámica estocástica de un MDP: dado un estado y una acción, el sistema transita hacia exactamente uno de varios estados siguientes posibles. Los autores demostraron además la equivalencia expresiva entre los LPADs y el Independent Choice Logic de Poole [19], consolidando a las ADs como el mecanismo más natural para representar efectos indeterminados en programas lógicos probabilísticos.

2.4. Compilación de conocimiento para inferencia eficiente

Para que la inferencia probabilística sobre programas con átomos de decisión sea computacionalmente tratable, sistemas como ProbLog emplean un flujo de compilación de conocimiento. El sistema transforma el programa lógico aterrizado en una fórmula proposicional que, a su vez, se compila en un circuito lógico. Sobre este circuito se ejecuta el Conteo de Modelos Ponderados (WMC). Darwiche y Marquis establecieron la taxonomía formal de los lenguajes de compilación e identificaron que estructuras como las d-DNNFs (*Deterministic Decomposable Negation Normal Forms*) permiten realizar inferencia exacta en tiempo lineal respecto al tamaño del circuito compilado [20]. Posteriormente, Darwiche introdujo los Diagramas de Decisión Sentencial (SDDs), una representación canónica que generaliza los clásicos BDDs y resulta estrictamente más sucinta [21].

Un avance directamente relevante para este trabajo es el algoritmo de diferenciación de circuitos propuesto por Darwiche [22], que permite calcular todas las derivadas parciales de un circuito d-DNNF en dos pasadas lineales sobre el mismo. Esta técnica amortiza el costo de inferencia cuando se requieren múltiples evaluaciones simultáneas, como ocurre durante la iteración de valor en MDP-ProbLog, donde cada par estado-acción demanda calcular las probabilidades de transición de todos los fluentes del siguiente estado. ProbLog2 integra estas estructuras como backends de compilación, ofreciendo flexibilidad para seleccionar entre d-DNNFs y SDDs según las compensaciones de espacio y tiempo del dominio [23]. Cualquier extensión a la capa de representación, como la integración de fluentes multivaluados propuesta, debe preservar la compatibilidad con este flujo de compilación para mantener las garantías de inferencia exacta.

2.5. Análisis Comparativo

La Tabla 4 compara los ocho enfoques revisados en este capítulo, agrupados por familia metodológica. Para cada uno se evalúan cuatro dimensiones: el tipo de variables de estado que soporta, la exactitud de la inferencia, la capacidad relacional del lenguaje, y el mecanismo de exclusión mutua.

Tabla 4: Comparativa de los enfoques del estado del arte.

Trabajo	**Variables de estado**	**Inferencia**	**Expresividad relacional**	**Exclusión mutua**
MDPs factorizados proposicionales				
Boutilier *et al.* [14]	Booleanas	Exacta	Proposicional	Manual
Boutilier, Dean y Hanks [7]	Booleanas	Exacta	Proposicional	Manual
Hoey *et al.* [8]	Booleanas	Exacta	Proposicional	Manual
Lenguajes de planificación				
Sanner [15]	Multivaluadas / continuas	Aprox.	Relacional	Nativa por tipo
Programación Lógica Probabilística				
De Raedt y Kimmig [16]	Booleanas	Exacta	Relacional	No aplica
Fierens *et al.* [3]	Booleanas	Exacta	Relacional	No aplica
Exclusión mutua y semántica causal				
Vennekens *et al.* [18]	Multivaluadas	Exacta	Relacional	Nativa
MDP-ProbLog				
Bueno *et al.* [6]	Booleanas	Exacta	Relacional	Manual
Propuesta	**Multivaluadas**	**Exacta**	**Relacional**	**Nativa**

Como se observa, la propuesta reúne simultáneamente las cuatro capacidades: variables de estado multivaluadas, inferencia exacta, expresividad relacional y exclusión mutua nativa.

2.6. Conclusión del Estado del Arte

Los MDPs factorizados demostraron que la planificación a escala es viable cuando los algoritmos explotan las independencias del dominio [14, 8]. Posteriormente, MDP-ProbLog combinó la expresividad relacional con la inferencia exacta para resolver MDPs de horizonte infinito [6]. Por su parte, los LPADs y CP-Logic formalizaron la exclusión mutua nativa como una propiedad semántica de los programas lógicos, la cual se alinea perfectamente con la dinámica de las transiciones estocásticas [13]. Finalmente, la compilación de conocimiento proporcionó las estructuras algorítmicas sobre las que este mecanismo evalúa probabilidades en tiempo lineal [20, 22].

La revisión de la literatura evidencia que la programación lógica ofrece herramientas probadas para el modelado multivaluado mediante Disyunciones Anotadas que MDP-ProbLog desaprovecha y cuya corrección conlleva una intervención en la arquitectura del sistema.

Dado que el espacio de estados deja de representarse como una estructura de factores exclusivamente binarios, el motor de resolución necesita adoptar un esquema que permita a cada variable operar bajo su cardinalidad original. Asimismo, el algoritmo de Iteración de Valor debe contemplar el cálculo del valor esperado como una ramificación (n-aria) para así evaluar correctamente las transiciones dictadas por las Disyunciones Anotadas.

Sobre esta base, se diseña e implementa la extensión descrita. El siguiente capítulo establece los fundamentos teóricos detallados que sustentan esta integración, comenzando por las bases de la programación lógica y culminando en las propiedades de los circuitos compilados que hacen computacionalmente posible la toma de decisiones.

3. Marco Teórico

El capítulo anterior estableció que MDP-ProbLog vincula la programación lógica probabilística con la resolución de MDPs factorizados. El objetivo de este capítulo es brindar los fundamentos teóricos necesarios para comprender tanto la arquitectura original del framework como los formalismos lógicos que sustentan nuestra extensión.

Se comienza fijando nociones básicas de programación lógica (Sección 3.2). Sobre esa base, se introduce la incertidumbre mediante la semántica de distribución y el lenguaje ProbLog (Secciones 3.3 y 3.4). Las Disyunciones Anotadas, presentadas en la Sección 3.5, incorporan la exclusión mutua como propiedad nativa del formalismo. La Sección 3.6 describe el Conteo de Modelos Ponderados y la compilación de conocimiento, mecanismos que convierten las consultas probabilísticas en un cálculo tratable.

Posteriormente, se formalizan los Procesos de Decisión de Markov y su versión factorizada (Secciones 3.7 y 3.8). Esta formalización permite cuantificar el costo de representar variables categóricas mediante fluentes booleanos. Sobre esta base, se presenta el framework MDP-ProbLog (Sección 3.9). El Capítulo 4 describe la extensión de su arquitectura para soportar fluentes multivaluados.

3.1. Grid Estocástico

Para ejemplificar los conceptos presentados a lo largo de este capítulo, utilizaremos el siguiente dominio de referencia en el que el entorno consiste en un grid estocástico de 2x3 adaptado del dominio clásico de Russell y Norvig, cuyas celdas identificamos mediante pares (r, c) con $r \in \{1, 2\}$ y $c \in \{1, 2, 3\}$.

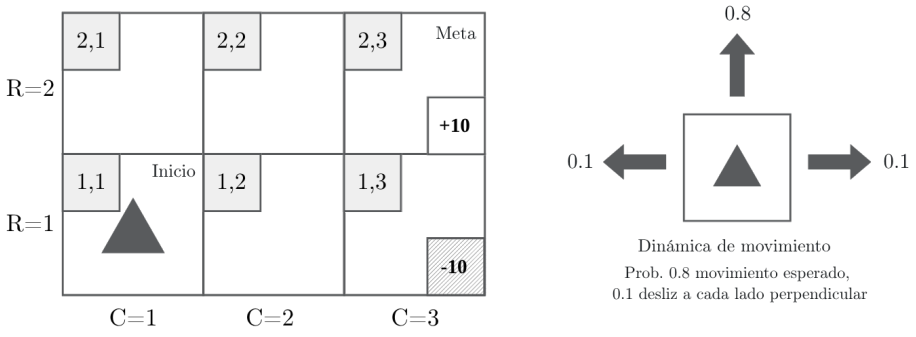

Figura 2: Grid de navegación 2x3 estocástico. Se incorporan transiciones probabilísticas.

La Figura 2 ilustra la configuración donde el agente inicia su recorrido en la celda (1,1) y busca alcanzar la meta para recibir una recompensa de +10. Se definen cuatro acciones

direccionales: norte, sur, este y oeste, donde cada transición incorpora incertidumbre.

Al ejecutar una acción, el agente avanza en la dirección deseada con una probabilidad del 0.8 y, con probabilidad de 0.1, se desliza hacia una dirección perpendicular. Toda acción actualiza la posición del agente, salvo cuando la celda de destino queda fuera del tablero, en ese caso, si el movimiento colisiona con un límite del entorno, el agente permanece en su posición actual.

3.2. Programación Lógica

Para representar formalmente sus estados y transiciones, MDP-ProbLog adopta el lenguaje de representación de la programación lógica. Esta sección establece la sintaxis del lenguaje, el proceso de resolución y la noción de instanciación que fundamentan la extensión probabilística de la sección siguiente.

Un programa lógico se construye a partir de hechos, reglas y consultas [24, 1]. Un hecho es una cláusula sin cuerpo que declara verdadero a un átomo. Un átomo es un predicado aplicado a sus argumentos. En el grid, el hecho `posicion(1,1).` afirma que el agente ocupa la celda $(1, 1)$. Una regla establece una implicación entre una cabeza y un cuerpo separados por el operador `:-`. La cabeza es verdadera solo cuando todos los literales del cuerpo son verdaderos.

La regla `meta_alcanzada :- posicion(2,3).` expresa que la condición de éxito se cumple si es que el agente ocupa la celda $(2, 3)$. Una consulta pregunta si un átomo es derivable a partir de un programa lógico definido. Si existe una prueba, el sistema devuelve la sustitución que lo hace verdadero, de lo contrario, falla.

Un programa lógico definido es un conjunto finito de cláusulas de Horn en el que cada cláusula tiene exactamente un átomo en la cabeza [24]. Las cláusulas pueden incluir variables escritas con mayúscula inicial, las cuales se interpretan como universalmente cuantificadas sobre el dominio de objetos. La regla `posicion(R,C) :- celda(R,C).` expresa que (R, C) es una posición válida para todo par que satisfaga `celda`. Esta capacidad de generalización permite describir espacios de estados extensos con un número reducido de cláusulas.

Para responder una consulta, Prolog emplea un proceso llamado resolución SLD [1] el cual opera de forma descendente sobre la lista de literales pendientes de la consulta. Dado un literal, el sistema busca una cláusula del programa cuya cabeza pueda unificarse con él.

Este proceso de unificación consiste en encontrar una sustitución de variables que haga idénticos a dos términos. Si se tiene éxito se aplica una sustitución y los literales unificados se reemplazan por el cuerpo de la cláusula seleccionada. Este proceso continúa recursivamente hasta que la lista queda vacía, considerandose una prueba exitosa, o hasta que ninguna cláusula puede unificarse con el literal actual.

El conjunto de todos los átomos base que pueden derivarse mediante unificaciones y resoluciones exitosas constituye precisamente el *modelo mínimo de Herbrand* del programa. Este

modelo es el conjunto más pequeño de átomos base que satisface todas las cláusulas del programa [24]. Por ejemplo, para el programa formado por el hecho `posicion(2,3).` y la regla `meta_alcanzada :- posicion(2,3).`, el modelo mínimo es {`posicion(2,3)`, `meta_alcanzada`}. En cambio, si el único hecho dl programa fuera `posicion(1,1).`, la regla no podría activarse y el modelo mínimo quedaría reducidoy a {`posicion(1,1)`}. De este modo, el modelo mínimo de Herbrand representa el conjunto exacto de consecuencias lógicas derivables del programa a través del proceso de resolución SLD.

Cuando un programa contiene variables, su instanciación produce el conjunto de todas las cláusulas base obtenidas al sustituir cada variable por una constante [1]. Este proceso se denomina *grounding* o instanciación y produce un programa aterrizado el cual contiene únicamente átomos base sin incluir variables. Un ejemplo de esto e la regla `celda(R,C) :- posicion(R,C).`, con $R \in \{1, 2\}$ y $C \in \{1, 2, 3\}$, produce en su instanciación seis cláusulas base, una para cada celda del dominio. Es así como el espacio de estados sobre el que opera el planificador se construye a partir de átomos base, aspecto que se retoma en la Sección 3.9. La sección siguiente introduce la incertidumbre extendiendo los hechos del programa con probabilidades.

3.3. Programación Lógica Probabilística

La Programación Lógica Probabilística (PLP) extiende el lenguaje de la sección anterior con una noción formal de incertidumbre. Para el lenguaje MDP-ProbLog, la PLP es la base sobre la que se calcula toda probabilidad de transición entre estados. Antes de describir cómo se realiza este cálculo, es necesario explicar qué significa exactamente que un programa lógico tenga una distribución de probabilidad asociada.

3.3.1. Semántica de distribución

Sato [2] observó que si se asigna una probabilidad independiente a cada hecho de un programa lógico, las reglas del programa propagan esa incertidumbre, induciendo una distribución sobre todos los modelos mínimos posibles. Esta idea, llamada *semántica de distribución*, es el fundamento común de la mayoría de los lenguajes de PLP modernos, incluido ProbLog.

Formalmente, un programa se representa como una tupla $DB = F \cup R$, donde $F = \{f_1, \ldots, f_k\}$ es el conjunto de hechos probabilísticos, cada uno asociado a una probabilidad $\theta_i \in [0, 1]$, y R es el conjunto de reglas determinísticas. Se asume que los hechos en F son estocásticamente independientes.

Una *realización total* L es una asignación completa de valores de verdad a todos los hechos de F, de modo que cada hecho f_i puede ser verdadero o falso. Por combinatoria, es fácil visualizar que, dado que existen k hechos probabilísticos y cada uno puede adoptar independientemente uno de dos valores de verdad, se tienen exactamente 2^k realizaciones totales posibles.

Es importante destacar que, para cada una de estas 2^k realizaciones L, su probabilidad se calcula multiplicando las probabilidades de los hechos que se consideran verdaderos por las probabilidades complementarias de aquellos que se consideran falsos:

$$P(L \mid DB) = \prod_{f_i \in L} \theta_i \prod_{f_i \in F \setminus L} (1 - \theta_i) \tag{1}$$

Cada realización total L determina un subconjunto concreto de hechos verdaderos que, al combinarse con las reglas determinísticas R, produce un programa lógico definido $L \cup R$ cuyo modelo mínimo de Herbrand es único, denotado $M_{DB}(L)$.

Por consiguiente, la distribución de probabilidad definida sobre las realizaciones induce naturalmente una distribución de probabilidad sobre los modelos mínimos. Cuando el conjunto F de hechos probabilísticos es finito, Sato demostró que toda la masa de probabilidad del programa se concentra exactamente en estos modelos derivados. La probabilidad de éxito de una consulta q se define entonces como la suma de las probabilidades de todas las realizaciones cuyo modelo mínimo satisface q:

$$P(q \mid DB) = \sum_{L : M_{DB}(L) \models q} P(L \mid DB) \tag{2}$$

Para aterrizar estos conceptos en el grid, considérese una situación donde el agente esté en la celda $(1, 1)$ ejecutando la acción `norte`. La dinámica establece una probabilidad de 80 % de avanzar hacia $(2, 1)$, mientras que el 20 % restante se reparte en desplazamientos ortogonales no deseados.

Dado que las herramientas para modelar transiciones con multiples resultados mutuamente excluyentes se introducirán más adelante, este primer acercamiento agrupa cualquier desvío en un único hecho probabilístico de fallo general. El Listado 1 muestra este fragmento inicial.

```
% Probabilidad agrupada de cualquier desvio ortogonal
0.2::fallo_norte.

% Consecuencias de ir hacia el norte estando en la pos(1,1)
pos(2,1) :- \+fallo_norte.
pos(1,1) :- fallo_norte.

% Consulta
query(pos(2,1)).
```

Código 1: Programa probabilístico. Se agrupan los desvíos en un único fallo.

Este programa define dos mundos posibles, correspondientes a las dos realizaciones del único hecho probabilístico. La Tabla 5 detalla ambas realizaciones con su probabilidad y el modelo mínimo resultante.

Tabla 5: Mundos posibles inducidos por la semántica de distribución sobre el Código 1.

`fallo_norte`	P(mundo)	`pos(2,1)`	Modelo mínimo
falso	0.8	verdadero	{`pos(2,1)`}
verdadero	0.2	falso	{`fallo_norte`, `pos(1,1)`}

La probabilidad de éxito de la consulta es la suma de las probabilidades de los mundos donde es verdadera, es decir, $P(\texttt{pos(2,1)}) = 0.8$. Este es exactamente el valor esperado y muestra cómo la incertidumbre del hecho probabilístico se propaga de manera determinista, a través de las reglas, hasta la consulta.

La ecuación 2 suma sobre 2^k realizaciones, donde k es el número de hechos probabilísticos del programa. Esta suma es de complejidad #P-hard en el caso general [3], por lo que no existe algoritmo polinomial conocido que la evalúe directamente para programas arbitrarios. Esta barrera computacional justifica la compilación de conocimiento como mecanismo de reducción de costo, descrita en la Sección 3.6.

Cabe aclarar que la semántica de distribución requiere un lenguaje concreto que la haga operacional. La sección siguiente presenta ProbLog, el lenguaje que implementa esta semántica.

3.4. Lenguaje ProbLog

ProbLog [1, 16] implementa la semántica de distribución con una extensión sintáctica mínima sobre Prolog. La extensión consiste en el *hecho probabilístico*, denotado `p::f.`, que declara que el átomo `f` es verdadero con probabilidad p de forma independiente a los demás hechos. Los átomos, las reglas, las consultas y la resolución SLD conservan el significado establecido en la Sección 3.2.

El motor de inferencia de ProbLog transforma un programa en una respuesta probabilística mediante cuatro etapas [3]. Primero, el programa se instancia reemplazando variables por constantes del dominio. Segundo, la fórmula resultante se traduce a una fórmula booleana ponderada. Tercero, esta fórmula se compila a un circuito lógico. Por último, la probabilidad de la consulta se obtiene evaluando el circuito mediante Conteo de Modelos Ponderados. Las etapas de compilación y evaluación recibirán su explicación detallada en la Sección 3.6.

Usando esta sintaxis, el Código 1 puede refinarse para representar con mayor fidelidad la dinámica del grid. El Código 2 expresa la transición completa para la acción norte desde la celda $(1, 1)$ como tres hechos probabilísticos seguidos de las reglas que determinan la celda destino en cada caso.

```
% Hechos probabilisticos independientes
0.8::exito_norte.
0.1::desliz_oeste.
0.1::desliz_este.
```

```
% Reglas de transicion
pos(2,1) :- exito_norte.
pos(1,1) :- desliz_oeste. % Choca y permanece en su lugar
pos(1,2) :- desliz_este.
```

Código 2: Transición para la acción norte desde $(1, 1)$ con hechos probabilísticos independientes.

En el ejemplo, Los tres hechos `exito_norte`, `desliz_oeste` y `desliz_este` se declaran como estocásticamente independientes. Sin embargo, las características del grid exigen que estos eventos sean mutuamente excluyentes. En cada transición, el agente puede avanzar hacia exactamente una celda. Al tratarlos como independientes, la semántica de distribución genera $2^3 = 8$ mundos posibles. Uno de esos mundos asigna valor verdadero a los tres hechos simultáneamente, con probabilidad $0.8 \times 0.1 \times 0.1 = 0.008$. Es así como se observa que existe un modelo mínimo donde se permiten las tres posiciones a la vez.

Para descartar estos mundos imposibles, el modelador debe agregar restricciones de integridad adicionales con la consecuencia de expandir el espacio de evaluación y oscurecer la descripción del modelo. La sección siguiente introduce el mecanismo que elimina este problema por construcción semántica.

3.5. Disyunciones Anotadas

Se ha comprobado que la representación de hechos probabilísticos independientes suele generar problemas cuando el modelador necesita representar una elección entre n resultados mutuamente excluyentes. Los 2^n mundos posibles incluyen combinaciones donde múltiples resultados son verdaderos simultáneamente, sin correspondencia con la dinámica real del sistema. Las Disyunciones Anotadas (*Annotated Disjunctions*, ADs), introducidas por Vennekens, Verbaeten y Bruynooghe [13], incorporan la exclusión mutua en la sintaxis de la cláusula. Esta modificación elimina los mundos espurios por construcción semántica.

Una Disyunción Anotada es una cláusula de la forma

$$(h_1 : p_1) \vee (h_2 : p_2) \vee \cdots \vee (h_n : p_n) \leftarrow b_1, \ldots, b_m \tag{3}$$

donde $h_1, \ldots, h_n$ son átomos, $p_1, \ldots, p_n$ son probabilidades con $\sum_{i=1}^{n} p_i \leq 1$, y $b_1, \ldots, b_m$ es el cuerpo de la cláusula [13]. Cuando la suma es estrictamente menor a uno, existe una probabilidad residual $1 - \sum_i p_i$ de que ningún head sea derivado. En transiciones de estado de un MDP la suma es igual a uno. Esta condición garantiza que cada instancia de la cláusula produzca exactamente un resultado.

La semántica de las ADs se define mediante el concepto de selección. Una selección σ asigna a cada cláusula AD del programa exactamente uno de sus heads. La instancia P_σ es el programa lógico definido que resulta de reemplazar cada cláusula AD por el hecho $h_{\sigma(r)}$ correspondiente a la cláusula r. La probabilidad de la selección σ es [13]

$$C_\sigma = \prod_r p_{\sigma(r)} \tag{4}$$

donde el producto recorre todas las cláusulas AD del programa. La distribución sobre interpretaciones se obtiene sumando C_σ sobre todas las selecciones cuya instancia P_σ deriva la interpretación objetivo.

El Código 3 reescribe la transición norte del grid de la Sección 3.4 como una única Disyunción Anotada. Esta cláusula genera exactamente tres selecciones. La Tabla 6 muestra la distribución resultante.

```
0.8::pos(2,1) ; 0.1::pos(1,1) ; 0.1::pos(1,2).
```

Código 3: Transición norte desde $(1, 1)$ representada como Disyunción Anotada.

Tabla 6: Distribución generada por la AD del Código 3. Cada selección corresponde a un único destino posible.

Selección σ	C_σ	Modelo mínimo
$\sigma \to$ `pos(2,1)`	0.8	{`pos(2,1)`}
$\sigma \to$ `pos(1,1)`	0.1	{`pos(1,1)`}
$\sigma \to$ `pos(1,2)`	0.1	{`pos(1,2)`}

En contraste con los ocho mundos de la Tabla 5, la AD genera tres mundos en los que exactamente un destino es verdaderom lo que permite que no exista ningún mundo con dos posiciones verdaderas simultáneamente. Laselección garantiza que en cada mundo exactamente un head de la disyunción sea derivado.

Los Programas Lógicos con Disyunciones Anotadas (LPADs) son expresivamente equivalentes a la Lógica de Elección Independiente (ICL) de Poole [19] y a CP-Logic [18]. Los tres formalismos definen la misma clase de distribuciones sobre interpretaciones de Herbrand. Sus diferencias residen en la notación empleada y en la interpretación causal que se asigna a las reglas.

ProbLog representa internamente cada Disyunción Anotada como un conjunto de nodos `choice` en una representación interna llamada ClauseDB, donde cada nodo . Cada nodo `choice` almacena el identificador del grupo AD al que pertenece y el átomo head asociado.[3] La sección siguiente presenta el mecanismo de inferencia que transforma los programas con ADs en evaluacionesde probabilidad de costo lineal.

3.6. Conteo de Modelos Ponderados y Compilación de Conocimiento

Aunque la sección define una distribución exacta sobre las interpretaciones de un programa, calcular la probabilidad de una consulta q bajo esta distribución sigue requiriendo sumar sobre todas las selecciones compatibles con q, por lo que esta suma presenta la misma complejidad #P-hard que la Ecuación 2 exhibe para los hechos independientes. El Conteo de Modelos Ponderados (*Weighted Model Counting*, WMC) convierte esta suma en una evaluación aritmética sobre una representación compacta del programa.

Primero, establezcamos que para poder aplicar WMC sobre un probgrama ProbLog, primero hay que convertirlo a una fórmula booleana ponderada donde se define una función de pesos w sobre los literales [3]. Para cada átomo probabilístico con probabilidad p, se asigna $w(l) = p$ al literal positivo y $w(\neg l) = 1 - p$ al negativo. Para los átomos derivados, ambos literales reciben peso 1. El peso de una interpretación ω es el producto de los pesos de todos sus literales:

$$W(\omega, w) = \prod_{l \in \omega} w(l) \tag{5}$$

El WMC de la fórmula φ bajo w es la suma de los pesos de todas sus interpretaciones satisfacibles:

$$\mathrm{WMC}(\varphi, w) = \sum_{\omega \models \varphi} W(\omega, w) \tag{6}$$

Fierens et al. demuestran que la probabilidad de evidencia de un programa ProbLog es igual al WMC de su fórmula ponderada [3]. Comparando las Ecuaciones 6 y 1, se observa que $W(\omega, w)$ coincide con $P(L \mid DB)$. Esta equivalencia transfiere el problema de inferencia probabilística al dominio aritmético de las fórmulas booleanas ponderadas.

El WMC de una fórmula arbitraria sigue siendo #P-duro. La compilación de conocimiento aborda este problema transformando la fórmula en una representación que admite WMC en tiempo lineal [20]. La representación objetivo es la Forma Normal de Negación Determinista y Descomponible (*d-DNNF*, por sus siglas en inglés). Una Forma Normal de Negación (NNF) es un grafo dirigido acíclico cuyos nodos internos son conjunciones o disyunciones y cuyos nodos hoja son literales. Una NNF es descomponible si los hijos de cada nodo conjuntivo no comparten átomos entre sí. Una NNF es determinista si los hijos de cada nodo disyuntivo son lógicamente inconsistentes entre sí [20]. Una d-DNNF satisface ambas propiedades.

La descomponibilidad permite computar el WMC de cada nodo conjuntivo como el producto de los WMC de sus hijos. El determinismo permite computar el WMC de cada nodo disyuntivo como la suma de los WMC de sus hijos. La propiedad de suavidad exige que todos los hijos de un nodo disyuntivo mencionen el mismo conjunto de átomos. Con estas tres propiedades

satisfechas, el recorrido ascendente sobre el circuito completo calcula WMC en $O(|C|)$, donde $|C|$ es el número de nodos del circuito [3, 20]. El circuito resultante, denominado circuito aritmético, puede compilarse una sola vez por programa y reutilizarse para evaluar todas las consultas posteriores sin recompilación.

Darwiche demostró que todas las derivadas parciales $\partial\,\mathrm{WMC}(\varphi, w)/\partial w(l)$ para cada literal l del circuito pueden calcularse simultáneamente mediante dos recorridos lineales [22]. El primero, ascendente, calcula el WMC y los valores intermedios de cada nodo. El segundo, descendente, propaga las contribuciones hacia los nodos hijos. La derivada $\partial\,\mathrm{WMC}/\partial w(l)$ es proporcional a la probabilidad marginal de l bajo la distribución del programa. Todas las probabilidades marginales se obtienen así en $O(|C|)$, sin necesidad de evaluaciones independientes del circuito.

ProbLog2 ofrece como alternativa al compilador d-DNNF el uso de Diagramas de Decisión Sentencial (SDDs), introducidos por Darwiche [21], los cuales son un subconjunto estricto de los d-DNNFs que imponen propiedades adicionales sobre la descomposición del espacio de variables. Un SDD con n variables y *treewidth* t tiene tamaño acotado por $O(n \cdot 2^t)$ [21]. Adicionalmente, los SDDs soportan combinación booleana en tiempo polinomial. Esta propiedad facilita la construcción incremental de circuitos cuando se añaden cláusulas al programa.

Durante la Iteración de Valor de MDP-ProbLog, cada respaldo de Bellman requiere evaluar $P(s' \mid s, a)$ para todos los posibles estados siguientes s' [6]. Estas evaluaciones corresponden a múltiples consultas WMC sobre el mismo circuito compilado. El algoritmo de diferenciación de Darwiche obtiene todas estas probabilidades en dos recorridos del circuito, independientemente del número de posibles estados siguientes. La compilación se realiza una sola vez y el circuito se reutiliza en cada iteración del algoritmo. La sección siguiente introduce los Procesos de Decisión de Markov, el marco sobre el cual MDP-ProbLog calcula políticas óptimas.

3.7. Procesos de Decisión de Markov

Los Procesos de Decisión de Markov (MDPs) formalizan la toma de decisiones secuencial bajo incertidumbre. Un agente observa el estado del entorno y selecciona una acción. La acción produce una recompensa y el entorno transita al siguiente estado según una distribución estocástica. El objetivo del agente es maximizar la suma de recompensas futuras descontadas. MDP-ProbLog utiliza este marco para calcular la política óptima de un agente en un entorno probabilístico.

Un MDP de horizonte infinito con factor de descuento es una tupla (S, A, T, R, γ), donde S es un conjunto finito de estados, A es un conjunto finito de acciones, $T : S \times A \times S \to [0, 1]$ es la función de transición estocástica con $\sum_{s' \in S} T(s, a, s') = 1$ para todo par (s, a), $R : S \times A \to \mathbb{R}$ es la función de recompensa inmediata y $\gamma \in [0, 1)$ es el factor de descuento [5]. El horizonte infinito con descuento es el marco sobre el que opera MDP-ProbLog optimiza sobre un futuro indefinido, pero las recompensas lejanas pesan menos que las inmediatas. La función de valor óptima $V^* : S \to \mathbb{R}$ satisface la ecuación de optimalidad de Bellman [4]:

$$V^*(s) = \max_{a \in A} \left[R(s,a) + \gamma \sum_{s' \in S} T(s,a,s') V^*(s') \right] \tag{7}$$

Esta ecuación establece que el valor óptimo de un estado es la recompensa inmediata máxima más el retorno descontado esperado del estado siguiente. La política óptima $\pi^* : S \to A$ elige en cada estado la acción que maximiza el lado derecho de la Ecuación 7. En el grid de la Sección 3.1, la acción óptima en cada celda depende tanto de la recompensa en la meta como de la distancia que falta para alcanzarla bajo el factor de descuento γ.

La Iteración de Valor (IV) calcula V^* de forma iterativa [5]. El algoritmo inicializa $V_0(s) = 0$ para todo s y aplica repetidamente el operador de Bellman:

$$V_{k+1}(s) = \max_{a \in A} \left[R(s,a) + \gamma \sum_{s' \in S} T(s,a,s') V_k(s') \right] \tag{8}$$

El algoritmo converge cuando el residuo máximo $\max_{s \in S} |V_{k+1}(s) - V_k(s)|$ cae por debajo del umbral $\varepsilon(1-\gamma)/\gamma$ [5]. La Figura 3 muestra la política π^* y los valores V^* del grid de la Sección 3.1 calculados con $\gamma = 0.9$ y $\varepsilon = 0.1$. Los valores son mayores en las celdas próximas a la meta en $(2,3)$ y decrecen con la distancia, reflejando el efecto acumulado del descuento.

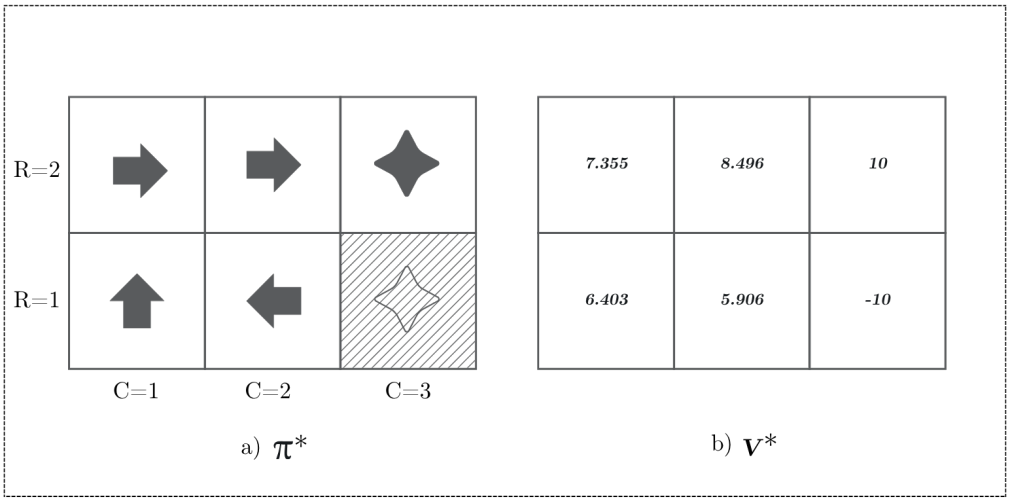

Figura 3: Política óptima: (a) una flecha indica la acción óptima $\pi^*(r,c)$; (b) muestra el valor $V^*(r,c)$ calculado por Iteración de Valor.

La Iteración de Valor opera sobre la enumeración explícita de S, por lo que la complejidad de cada respaldo es $O(|S|^2 \cdot |A|)$. A medida que el número de variables de estado crece, $|S|$ crece exponencialmente y el algoritmo deja de ser aplicable directamente. La sección siguiente examina cómo la representación factorizada aborda esta limitación.

3.8. MDPs Factorizados y Fluentes de Estado

Un MDP en el que el estado se describe mediante n variables tiene un espacio de estados de tamaño $\prod_{i=1}^{n} |\mathrm{dom}(X_i)|$. Para n variables binarias, $|S| = 2^n$. Para dominios con decenas de variables, la enumeración explícita de S resulta computacionalmente prohibitiva. Los MDPs Factorizados responden a este problema representando los estados y las transiciones mediante la estructura interna de las variables de estado [14].

En un MDP Factorizado, el estado s es una valuación del vector de variables $\mathbf{X} = (X_1, \ldots, X_n)$, donde cada X_i tiene dominio finito $\mathrm{dom}(X_i)$ [14]. La función de transición se factoriza de acuerdo con la estructura de una Red Bayesiana Dinámica (DBN). Para cada variable X_i, el valor siguiente X_i' depende únicamente de un subconjunto de variables padre $\mathrm{Pa}(X_i') \subseteq \mathbf{X}$:

$$P(s' \mid s, a) = \prod_{i=1}^{n} P\big(X_i' \mid \mathrm{Pa}(X_i'),\, a\big) \tag{9}$$

Esta factorización expresa la dinámica del sistema en términos de dependencias locales. El número de parámetros necesarios crece linealmente con el número de variables en lugar de cuadráticamente con $|S|$.

En MDP-ProbLog, las variables del MDP Factorizado se denominan fluentes de estado. Un fluente de estado es una variable con un dominio discreto finito [6]. Un fluente booleano tiene dominio $\{0, 1\}$. Un fluente multivaluado tiene dominio $\{v_1, \ldots, v_k\}$ con $k \geq 2$.

Una variable categórica con k valores se codifica mediante $\lceil \log_2 k \rceil$ fluentes booleanos. Este esquema genera $2^{\lceil \log_2 k \rceil}$ combinaciones binarias, de las cuales solo k son válidas. Las combinaciones restantes son estados espurios sin correspondencia en el dominio real. Para el fluente de columna $c \in \{1, 2, 3\}$ del grid, la codificación booleana requiere $\lceil \log_2 3 \rceil = 2$ bits, generando $2^2 = 4$ combinaciones. Solo tres corresponden a columnas reales; la cuarta es un estado espurio. El espacio de estados booleano completo del grid contiene 8 estados, de los cuales 6 son físicamente posibles y 2 son espurios, como se analizó en la Introducción.

Los estados espurios incrementan el espacio enumerado por el solver, el tamaño del circuito compilado y el costo de cada respaldo de Bellman. La codificación mixed-radix, en la que cada fluente conserva su cardinalidad natural, elimina estos estados espurios al representar $r \in \{1, 2\}$ con base 2 y $c \in \{1, 2, 3\}$ con base 3, produciendo $2 \times 3 = 6$ estados reales. La sección siguiente describe cómo MDP-ProbLog integra el formalismo de los MDPs Factorizados con la programación lógica probabilística.

3.9. El framework MDP-ProbLog

MDP-ProbLog integra el formalismo de los MDPs Factorizados con el lenguaje ProbLog para resolver problemas de planificación probabilística de horizonte infinito. La integración se basa en la correspondencia entre la evaluación de un programa ProbLog bajo evidencia y el respaldo de Bellman sobre un estado del MDP [6]. El usuario especifica el modelo declarativamente en ProbLog. El solver transforma esa especificación en una política óptima mediante Iteración de Valor sobre el circuito compilado.

Un programa MDP-ProbLog extiende la sintaxis de ProbLog con tres categorías de predicados reservados [6]. El predicado `state_fluent/1` declara los fluentes de estado cuya valuación define un estado del MDP. El predicado `action/1` declara las acciones disponibles en cada estado. El predicado `utility/2` especifica la función de recompensa, asociando un átomo con un valor numérico.

La dinámica del MDP se especifica mediante un programa de dos instantes temporales. Los átomos etiquetados con el marcador $t = 0$ representan el estado actual y la acción ejecutada, a su vez, los átomos etiquetados con $t = 1$ representan el estado siguiente. Las reglas de transición relacionan el estado en $t = 0$ con el estado en $t = 1$ a través de hechos probabilísticos auxiliares que codifican la incertidumbre de las acciones.

El Código 4 enseña la estructura de un programa MDP-ProbLog para el grid de la Sección 3.1 con codificación booleana. Los tres fluentes booleanos `row2(0)`, `col2(0)` y `col3(0)` codifican la posición del agente mediante tres bits. Este esquema produce $2^3 = 8$ combinaciones, de las cuales seis son válidas y dos son espurias, como se describió en la Sección 3.8.

```
% Fluentes de estado: codificacion booleana de la posicion del
   agente
% (3 bits generan 2^3 = 8 combinaciones; solo 6 corresponden a
   celdas reales)
state_fluent(row2(0)).    % fila 2 si verdadero, fila 1 si falso
state_fluent(col2(0)).    % columna 2 si verdadero
state_fluent(col3(0)).    % columna 3 si verdadero

% Acciones
action(norte). action(sur). action(este). action(oeste).

% Incertidumbre estocastica para la accion norte
0.8::avance_n(0).   0.1::desvio_e_n(0).   0.1::desvio_o_n(0).

% Transicion: avance hacia la fila 2 con accion norte (regla
   representativa)
row2(1) :- norte(0), avance_n(0), \+ row2(0).
% ... (reglas de transicion para col2, col3 y las demas acciones)

% Recompensa: +10 al alcanzar la celda meta (fila 2, columna 3)
meta(1) :- row2(1), col3(1), \+ col2(1).
```

```
utility(meta(1), 10).
```

Código 4: Estructura de un programa MDP-ProbLog para el grid 2×3 con codificación booleana.

Bueno et al. demuestran que evaluar un programa MDP-ProbLog bajo evidencia (s, a) produce exactamente las probabilidades de transición $P(s' \mid s, a)$ requeridas por el respaldo de Bellman [6]. La evaluación se realiza mediante WMC sobre el circuito compilado. El resultado es equivalente al lado derecho de la Ecuación 7. Esta equivalencia garantiza que el algoritmo de IV de MDP-ProbLog calcula la misma política óptima que la IV clásica.

El solver de MDP-ProbLog ejecuta tres etapas [6]. En la etapa de preprocesamiento, se resuelven las reglas intencionales y se construye el programa aumentado con los predicados de función de valor. En la etapa de compilación, el programa aterrizado se convierte a una fórmula booleana ponderada y se compila a un circuito d-DNNF. En la etapa de Iteración de Valor, el solver fija la evidencia (s, a) para cada par estado-acción, evalúa el circuito mediante WMC y actualiza V hasta convergencia.

La restricción booleana de la versión original reside en la etapa de preprocesamiento. Cada fluente de estado se inyecta como el hecho probabilístico `0.5::f(0)`, que presupone un dominio binario $\{0, 1\}$. Esta decisión impide representar directamente variables categóricas sin codificación binaria manual. La extensión propuesta interviene exclusivamente en esta capa de inyección. Para los fluentes multivaluados, sustituye los hechos probabilísticos independientes por Disyunciones Anotadas. El resto del pipeline permanece sin modificaciones.

4. Diseño e Implementación

Una vez establecidos los fundamentos teóricos de la Programación Lógica Probabilística, los Procesos de Decisión de Markov factorizados y las Disyunciones Anotadas, el presente capítulo desarrolla el diseño y la construcción de la extensión al núcleo de MDP-ProbLog. Se analiza primero la arquitectura original del sistema para identificar sus limitaciones estructurales en la representación del espacio de estados. A partir de esta evaluación, se detalla el diseño de las nuevas estructuras de datos y los mecanismos de inferencia de tipos, seguido por las modificaciones al motor de inferencia y al algoritmo de resolución. Finalmente, se demuestra cómo la extensión preserva la retrocompatibilidad con los modelos booleanos preexistentes.

4.1. Análisis de la arquitectura original

Para determinar los puntos de intervención de la extensión propuesta, es necesario examinar cómo el framework MDP-ProbLog procesaba la información en su diseño original [6]. El sistema opera bajo una arquitectura modular que actúa como capa intermedia entre el motor de inferencia probabilística de ProbLog y un algoritmo de resolución numérica.

4.1.1. Componentes del sistema original

La arquitectura original se compone de cuatro módulos con responsabilidades delimitadas:

`engine.py` Implementa un adaptador sobre el motor interno de ProbLog (`DefaultEngine`). Encapsula la Base de Datos de Cláusulas (*ClauseDB*) y expone métodos para añadir hechos lógicos, ejecutar el grounding relevante y compilar circuitos evaluables para el Conteo de Modelos Ponderados.

`fluent.py` Define las abstracciones del estado del mundo. Las clases `Fluent` y `StateSpace` generan e indexan las configuraciones posibles del sistema, asumiendo un espacio factorizado puramente booleano.

`mdp.py` Actúa como entidad coordinadora del framework. Interroga la base de datos de cláusulas buscando predicados reservados (`state_fluent/1`, `action/1`, `utility/2`), coordina la incorporacion de hechos y traduce las relaciones lógicas en una semántica de transiciones de Markov.

`value_iteration.py` Implementa el algoritmo de Iteración de Valor. Opera sobre índices enteros y distribuciones marginales para aproximar la ecuación de optimalidad de Bellman (Ecuación **??**).

4.1.2. Flujo de ejecución

La interacción entre estos componentes sigue un flujo secuencial de cinco fases:

1. **Análisis y carga.** El módulo `engine.py` realiza el *parsing* del archivo Prolog proporcionado por el usuario y lo inyecta en la `ClauseDB`.

2. **Descubrimiento del dominio.** El coordinador (`mdp.py`) interroga la base de datos para identificar los fluentes de estado y las acciones, e instancia los iteradores correspondientes en `fluent.py`.

3. **Inyección temporal.** Para modelar la dinámica secuencial, `mdp.py` transforma las declaraciones abstractas agregando estampas de tiempo ($t = 0$ para el estado actual, $t = 1$ para el siguiente estado). Simultáneamente, introduce hechos probabilísticos neutrales en $t = 0$ para permitir la sustitución posterior de evidencia durante la evaluación.

4. **Aterrizado y compilación.** Se ejecuta un *grounding* relevante respecto a las consultas de interés. Las reglas de transición se compilan en un circuito proposicional tratable (SDD o d-DNNF).

5. **Resolución.** El módulo `value_iteration.py` recorre el espacio de estados, inyecta cada par estado-acción como evidencia determinista en el motor, y solicita el cálculo de las probabilidades marginales $P(s' \mid s, a)$ mediante WMC para actualizar la función de valor $V(s)$.

Esta separación de responsabilidades facilitó la tarea de realizar modificaciones y constuir sobre el un solver funcional y matemáticamente correcto.

4.1.3. Limitaciones en la gestión de fluentes

Aunque MDP-ProbLog permite modelar variables de estado relacionales mediante el predicado `state_fluent/1`, su diseño interno asume que cada instancia aterrizada (*ground atom*) opera como una variable de Bernoulli estocásticamente independiente.

El módulo `fluent.py` original implementaba la clase `StateSpace` como un iterador combinatorio plano. Dado un conjunto de n fluentes booleanos, el sistema generaba el producto cartesiano completo, produciendo un espacio de estados de tamaño 2^n. Esta arquitectura resulta funcional para dominios puramente binarios, pero genera un desajuste al modelar fenómenos con estados representados con variables categoricas o multivaluadas.

Esta divergencia entre la semántica del dominio y la representación computacional genera dos consecuencias negativas. Por un lado, delega en el usuario la responsabilidad de escribir reglas lógicas adicionales para bloquear transiciones hacia los estados inválidos, lo que aumenta la complejidad del modelado y la propensión a errores. Por otro lado, el algoritmo de Iteración de Valor calcula respaldos de Bellman para estados inalcanzables, consumiendo recursos computacionales sin contribuir a la solución.

4.1.4. Identificación de puntos de extensión

El análisis de las limitaciones expuestas revela que la solución no requiere alterar el motor de inferencia probabilística subyacente de ProbLog. La compilación a circuitos lógicos y el WMC permanecen como la estrategia de evaluación del framework. La intervención se localiza en la capa intermedia donde las declaraciones lógicas se transforman en variables de estado evaluables para el solver. Se identificaron tres puntos de extensión:

1. **Representación del espacio de estados.** Reemplazar la representación por una estructura factorizada que admita factores de cardinalidad arbitraria.

2. **Inyección de hechos en la ClauseDB.** Adaptar el mecanismo de inyección temporal para que los fluentes multivaluados se representen como Disyunciones Anotadas en lugar de hechos probabilísticos independientes.

3. **Cálculo del valor esperado.** Reestructurar la evaluación del valor futuro en la Iteración de Valor para que recorra los factores del esquema en lugar de enumerar combinaciones binarias.

Al acotar las modificaciones a estos tres puntos, se preservan las garantías de inferencia exacta de ProbLog. La siguiente sección detalla el diseño formal de cada componente de la extensión.

4.2. Diseño de la extensión

Esta sección detalla el diseño conceptual de la extensión propuesta. El objetivo es transitar de un modelo de variables booleanas independientes hacia una estructura factorizada que soporte fluentes de estado multivaluados de forma nativa. Para lograrlo, se diseñaron dos componentes centrales: una estructura de datos factorizada (`FluentSchema`) y un mecanismo de clasificación automática (`FluentClassifier`).

4.2.1. FluentSchema: estructura factorizada del espacio de estados

El primer componente del rediseño reemplaza el almacenamiento de fluentes basado en listas planas por una estructura de datos centralizada denominada `FluentSchema`. Esta clase define el espacio de estados como el producto cartesiano de factores lógicamente independientes, donde cada factor pertenece a una de dos categorías:

- **Booleano (base 2).** Una variable binaria independiente que toma valores en $\{0, 1\}$. Corresponde a los fluentes tradicionales de MDP-ProbLog.

- **Multivaluado (base N).** Un grupo de N opciones mutuamente excluyentes donde exactamente una está activa en cada estado. La codificación interna utiliza representación *one-hot* sobre las opciones del grupo.

El esquema mantiene internamente tres estructuras de datos: la lista ordenada de factores (donde cada factor es una lista de términos atemporales), la lista de bases correspondientes, y

una lista aplanada de todos los términos en orden de registro. Los strides para la indexación mixed-radix se calculan bajo demanda y se almacenan en caché.

Para gestionar los factores multivaluados, el esquema implementa un mecanismo de agrupamiento basado en el nombre del predicado (*functor*). Al instanciar el programa, el sistema agrupa los términos multivaluados evaluando su functor. Esto garantiza, por ejemplo, que los valores `pos(a)`, `pos(b)` y `pos(c)` se agrupen como alternativas excluyentes del mismo factor. Esta convención constituye una de las limitaciones documentadas en la sección de Alcances (1.5.2): todos los valores de un fluente multivaluado deben compartir el mismo nombre de predicado.

4.2.2. FluentClassifier: mecanismo de clasificación automática

Para que el esquema factorizado se construya correctamente sin obligar al usuario a declarar manualmente el tipo de cada fluente en todos los casos, se diseñó el módulo `FluentClassifier`. Este componente ejecuta un proceso de clasificación en tres etapas: validación estática de las declaraciones, registro e inferencia de tipos, y ensamblaje del esquema.

El clasificador admite dos modos de declaración complementarios:

- **Modo implícito (`state_fluent/1`):** El *framework* infiere automáticamente el tipo del fluente analizando el origen de sus valores aterrizados (*grounded values*).
- **Modo explícito (`state_fluent/2`):** El usuario especifica directamente una etiqueta de tipo (`bool` o `multivalued`), otorgando control total sobre la clasificación e ignorando la inferencia automática.

Algoritmo 1 Inferencia de tipo por intersección de orígenes estocásticos

Entrada: Conjunto de términos aterrizados G de aridad a; índice invertido $\mathcal{I} : \Sigma \to \mathcal{P}(\Gamma)$ que mapea valores Σ a conjuntos de grupos AD Γ.
Salida: Clasificación de tipo: `bool` o `multivalued`.

1: **if** $a = 0$ **then return** `bool` ▷ Los fluentes proposicionales son siempre booleanos.
2: **end if**
3: **for** $p \leftarrow 1$ **a** a **do**
4: $V_p \leftarrow \{\arg_p(g) \mid g \in G\}$ ▷ Extraer conjunto de valores en la posición p.
5: $C \leftarrow \bigcap_{v \in V_p} \mathcal{I}(v)$ ▷ Intersección de grupos AD para todos los valores.
6: **if** $C \neq \emptyset$ **then return** `multivalued` ▷ Todos los valores provienen de la misma AD.
7: **end if**
8: **end for**
return `bool` ▷ Ninguna posición proviene íntegramente de una AD.

Durante el análisis estático, el clasificador aplica reglas de validación estrictas para prevenir errores de modelado. Por ejemplo, si un grupo multivaluado resulta con una cardinalidad inferior a dos opciones, se interrumpe la ejecución con un `FluentCardinalityError`, ya que

un grupo mutuamente excluyente requiere al menos dos estados válidos para contribuir al espacio mixed-radix. Si un fluente se declara en ambos modos, el sistema emite una advertencia de duplicidad y prioriza la declaración explícita.

Reglas de inferencia implícita. En el modo implícito, la decisión de clasificación depende directamente de la aridad del término lógico y de la estructura de la Base de Datos de Cláusulas:

1. **Aridad 0:** Los fluentes proposicionales carecen de argumentos para generar un dominio de valores; por lo tanto, siempre se clasifican como booleanos .

2. **Aridad ≥ 1:** El clasificador verifica si los valores aterrizados en alguna posición argumental provienen íntegramente de una Disyunción Anotada (AD). Si la intersección de estos valores con el vocabulario de alguna AD declarada es exhaustiva, el fluente se clasifica como `multivalued`; en caso contrario, si provienen de hechos estáticos ordinarios, se clasifica como `bool`.

El Algoritmo 1 formaliza esta lógica de decisión por intersección.

Agrupamiento y el problema del Colapso Estructural. Una vez que un conjunto de términos implícitos es identificado como multivaluado, el sistema consolida el factor basándose exclusivamente en el nombre del predicado (*functor*) . Bajo esta convención, el framework interpreta los fluentes como una única variable categórica global por cada nombre de predicado.

Si bien este comportamiento es eficiente para dominios simples, introduce un riesgo documentado como *Colapso Estructural* . Este fenómeno ocurre cuando el modelador intenta representar familias de variables indexadas por entidad (por ejemplo, `pos(robot1, a)` y `pos(robot2, b)`) utilizando el modo implícito. Al compartir el mismo *functor* (`pos`), el clasificador las agrupa erróneamente dentro de un único factor mutuamente excluyente, destruyendo la independencia estocástica entre los distintos robots e invalidando la semántica del dominio .

Justificación del modo explícito. Es precisamente el riesgo de colapso estructural lo que fundamenta la necesidad arquitectónica del predicado `state_fluent/2` con la etiqueta `multivalued`. El modo explícito desactiva el agrupamiento ciego por functor, permitiendo al modelador definir reglas parametrizadas que instancien múltiples factores multivaluados verdaderamente independientes. Esta dualidad de diseño garantiza que el framework mantenga una sintaxis ligera para variables globales, sin sacrificar la expresividad relacional necesaria para dominios multi-agente complejos.

4.2.3. Codificación *mixed-radix* del espacio de estados

Con los fluentes correctamente clasificados y agrupados en el `FluentSchema`, el sistema debe iterar y enumerar el nuevo espacio de estados sin generar combinaciones inválidas. En la

arquitectura original, la enumeración utilizaba un contador binario que asignaba un índice entero a cada combinación asumiendo base 2 para todos los factores. Al introducir variables con cardinalidad mayor a 2, se requiere un sistema de numeración capaz de manejar bases heterogéneas.

La clase `FactorSpace` (de la cual heredan `StateSpace` y `ActionSpace`) implementa una codificación de base mixta (*mixed-radix*). Bajo este esquema, el espacio de estados se define como el producto cartesiano de m factores independientes, donde cada factor k posee una base b_k correspondiente a su cardinalidad.

El tamaño total del espacio de estados se define como el producto de todas las bases:

$$|\mathcal{S}| = \prod_{k=1}^{m} b_k \tag{10}$$

Para mapear cada estado a un índice entero $I \in [0, |\mathcal{S}| - 1]$, se calcula un stride S_k para cada factor, definido como el producto de las bases de los factores precedentes:

$$S_k = \prod_{i=1}^{k-1} b_i \quad \text{con } S_1 = 1 \tag{11}$$

Dada una configuración donde cada factor k toma un valor local $v_k \in [0, b_k - 1]$, el índice global se calcula como:

$$I = \sum_{k=1}^{m} (v_k \times S_k) \tag{12}$$

La operación inversa (decodificación de un índice a una valuación) se realiza extrayendo sucesivamente el residuo módulo la base de cada factor:

$$v_k = I \text{ mód } b_k, \quad I \leftarrow \left\lfloor \frac{I}{b_k} \right\rfloor \tag{13}$$

Ambas operaciones eliminan la generación de estados imposibles, ya que el espacio iterado corresponde exactamente a las combinaciones semánticamente válidas del modelo.

4.2.4. Sintaxis de usuario para fluentes multivaluados

El propósito último de reestructurar la representación interna del *framework* es proporcionar al modelador una interfaz declarativa más natural, robusta y matemáticamente segura. La extensión permite declarar variables categóricas y especificar sus transiciones estocásticas directamente mediante la sintaxis nativa de las Disyunciones Anotadas (AD) de ProbLog, eliminando la necesidad de gestionar la exclusión mutua de forma manual.

Para ilustrar el impacto de este rediseño a nivel de código, considérese un dominio [Pensar bien que dominio presentar, algo sencillo]

Código 5: Modelado de posición en la arquitectura original.

En el Código 5, la semántica del dominio diverge severamente de su representación computacional. Al tratar cada posición como una variable de Bernoulli independiente, el motor de inferencia asume un espacio de estados de tamaño $2^4 = 16$ combinaciones, de las cuales solo 4 son semánticamente válidas . El modelador asume la carga de escribir reglas lógicas adyacentes para asegurar que la probabilidad fluya correctamente y evitar transiciones hacia configuraciones imposibles (por ejemplo, el agente existiendo simultáneamente en `pos(a)` y `pos(b)`).

Con la extensión desarrollada, el mismo modelo se especifica explotando la semántica de las Disyunciones Anotadas.

```
% 1. Declaracion unificada (Genera 4 estados bajo codificacion
   one-hot)
state_fluent(pos(X), multivalued) :- loc(X).
loc(a). loc(b). loc(c). loc(d).

% 2. Transicion estocastica nativa (Garantiza suma de
   probabilidades = 1.0)
0.8::pos(b, 1); 0.2::pos(a, 1) :- pos(a, 0), move_right.

% 3. (No se requieren reglas de exclusion mutua)
```

Código 6: Modelado de posición utilizando fluentes multivaluados.

Como se observa en el Código 6, la nueva arquitectura permite una declaración explícita parametrizada que colapsa las cuatro posiciones en un único factor de cardinalidad 4. Las reglas de transición codifican directamente la dinámica de Markov utilizando las cabezas múltiples de la Disyunción Anotada.

Este enfoque presenta tres ventajas arquitectónicas fundamentales:

1. **Precisión dimensional:** El espacio de estados se reduce de 2^N a N combinaciones por factor, iterando exclusivamente sobre configuraciones válidas mediante la codificación *mixed-radix*.

2. **Seguridad probabilística:** La restricción estocástica $\sum \alpha_i = 1$ es garantizada por construcción por el analizador de ProbLog para las ADs.

3. **Mantenibilidad:** Se elimina la necesidad de redactar y depurar reglas manuales de exclusión mutua, reduciendo drásticamente la verbosidad del programa lógico.

4.3. Implementación

Esta sección describe las modificaciones realizadas sobre los módulos del framework para materializar el diseño expuesto en la sección anterior. Las adaptaciones se concentran en tres componentes: el adaptador al motor de inferencia, el coordinador central del MDP, y el algoritmo de resolución.

4.3.1. Adaptador al motor de inferencia

La clase `Engine` actúa como la interfaz entre MDP-ProbLog y el núcleo de ProbLog. Para soportar fluentes multivaluados, se incorporaron dos métodos nuevos.

Inyección de Disyunciones Anotadas. El método `add_annotated_disjunction` permite añadir dinámicamente una AD en la `ClauseDB`. Recibe un grupo de hechos mutuamente excluyentes junto con sus probabilidades, construye internamente un objeto `AnnotatedDisjunction` de ProbLog, y lo incorpora a la base de datos. El método retorna los identificadores de los nodos `choice` resultantes, que representan las ramas estocásticas de la disyunción dentro del circuito.

Construcción del índice invertido. El método `get_ads_inverted_index` recorre linealmente la tabla de instrucciones de la `ClauseDB`, identificando todos los nodos de tipo `choice`. Para cada nodo detectado, extrae el término lógico contenido en la estructura interna del nodo y registra sus argumentos en un diccionario que mapea cada valor concreto (como cadena de texto) al conjunto de identificadores de grupo de la AD que lo generó. El resultado es un índice invertido con la estructura:

$$\{\texttt{valor_string} \rightarrow \{\texttt{group_id_1, group_id_2, ...}\}\}$$

Este índice es consumido por el `FluentClassifier` durante la fase de inferencia de tipos, como se describió en 4.2.2. La construcción del índice se apoya en atributos internos de los nodos `choice` de ProbLog (específicamente `node.functor.args[2]` y `node.group`), lo cual constituye una dependencia sobre la representación interna de ProbLog documentada como limitación conocida.

4.3.2. Coordinador central del MDP

La clase `MDP` coordina el flujo completo que transforma el programa lógico del usuario en un problema de optimización resoluble. Para integrar el esquema factorizado, el método de inicialización `__prepare()` se reestructuró en cinco etapas secuenciales:

1. **Clasificación de fluentes.** Se instancia el `FluentClassifier`, que consume las declaraciones del programa y el índice invertido de ADs para construir el `FluentSchema`.

2. **Inyección del estado actual ($t = 0$).** Para cada factor del esquema, se inyectan hechos que representan el estado actual. Los factores booleanos se inyectan como hechos probabilísticos con probabilidad neutral de 0.5 mediante `add_fact`. Los factores

multivaluados se inyectan como Disyunciones Anotadas con probabilidades uniformes $1/N$ mediante `add_annotated_disjunction`. En ambos casos, las probabilidades son neutrales y serán sustituidas por los valores de evidencia (0 o 1) durante la evaluación.

3. **Inyección de acciones.** Las acciones declaradas en el programa se inyectan como una única Disyunción Anotada con distribución uniforme, siguiendo el mismo principio de exclusión mutua nativa.

4. **Grounding relevante.** Se construye la lista de consultas combinando las utilidades, los fluentes de siguiente estado ($t = 1$), las acciones y los fluentes del estado actual ($t = 0$). La inclusión de los fluentes $t = 0$ en esta lista es una decisión técnica que garantiza que ProbLog preserve sus identificadores durante el proceso de optimización del grounding, permitiendo localizarlos posteriormente al sustituir evidencia.

5. **Compilación.** Los fluentes de siguiente estado y las utilidades se compilan en nodos del circuito evaluable, generando los mapas término→nodo que se reutilizan en cada evaluación posterior.

4.3.3. Adaptaciones al algoritmo de resolución

La adopción de una representación mixed-radix exige que el cálculo del valor esperado futuro durante las actualizaciones de Bellman sea capaz de navegar la estructura factorizada. En la implementación original, este cálculo enumeraba las combinaciones binarias posibles. La nueva implementación ejecuta un recorrido recursivo sobre los factores del esquema.

Algoritmo 2 Cálculo Expected Value

Entrada: Profundidad k, índice global acumulado I, probabilidad conjunta P_{joint}.
Entrada: Secuencia de factores estructurados $\mathcal{F}$, vector de strides S, función de valor V.
Entrada: Umbral de filtrado probabilístico ϵ.
Salida: Valor esperado escalar E para el subárbol.

```
function ExpectedValue(k, I, P_joint)
    if k = |F| then                    ▷ Caso base: Se han procesado todos los factores. return P_joint × V[I]
    end if
    E ← 0
    for cada tupla (v, p) ∈ F_k do                ▷ Iterar ramas activas del factor actual.
        if p > ε then                                  ▷ Filtrado de ramas irrelevantes .
            i_local ← IndexLocal(F_k, v)               ▷ Decodificación del índice local.
            I_next ← I + (i_local × S_k)               ▷ Acumulación posicional mixed-radix.
            P_next ← P_joint × p                       ▷ Acumulación de probabilidad conjunta.
            E ← E + ExpectedValue(k + 1, I_next, P_next)   ▷ Llamada recursiva.
        end if
    end for
    return E
end function
```

En cada nivel de recursión k, correspondiente a un factor del `FluentSchema`, el algoritmo extrae las ramas activas del factor (los posibles valores futuros con sus probabilidades marginales). Para cada rama, calcula el índice global del estado sucesor acumulando el producto del índice local por el stride posicional del factor k, y multiplica la probabilidad conjunta del camino. Al alcanzar el nivel base (todos los factores procesados), el índice acumulado identifica unívocamente el estado destino y se retorna el producto de la probabilidad conjunta por el valor almacenado $V[\text{índice}]$.

Adicionalmente, el método `structured_transition` del MDP aplica un filtro previo sobre cada factor, descartando las ramas cuya probabilidad sea inferior a un umbral configurable (10^{-6} por defecto). Esto reduce el número de ramas activas por factor y, en consecuencia, la profundidad efectiva del árbol recursivo. En transiciones deterministas o cuasi-deterministas, donde un solo valor concentra la totalidad de la masa probabilística, el recorrido se reduce a una sola rama por factor.

4.3.4. Herramientas de verificación y exportación

Para facilitar la verificación de los modelos resueltos, se desarrollaron dos módulos complementarios.

El `CSVExporter` permite exportar las estructuras del MDP y los resultados de la Iteración de Valor a archivos CSV. Las exportaciones disponibles incluyen la matriz de transición $P(s' \mid s, a)$ expandida a partir de las transiciones factorizadas, la función de recompensa $R(s, a)$, la función de valor óptima $V^*(s)$, la política óptima $\pi^*(s)$, la tabla de valores Q (si fue calculada), y el historial de convergencia (si fue registrado). La expansión de las transiciones factorizadas a formato plano se realiza mediante una recursión análoga a la del cálculo del valor esperado, generando el producto cartesiano de las ramas activas por factor.

El `Simulator` permite ejecutar trayectorias de simulación a partir de un estado inicial, siguiendo la política calculada. En cada paso de tiempo, el simulador selecciona la acción prescrita por la política, obtiene la distribución de transición factorizada del motor, y muestrea el siguiente estado mediante selección categórica ponderada sobre cada factor del esquema.

Ambas herramientas operan sobre la misma representación factorizada que utiliza el solver, garantizando consistencia entre la resolución y la inspección del modelo.

4.4. Retrocompatibilidad

La extensión fue diseñada para preservar el funcionamiento de los modelos booleanos existentes sin requerir modificaciones por parte del usuario. Esta garantía se sustenta en el comportamiento del `FluentClassifier` ante programas que no contienen Disyunciones Anotadas.

Cuando el clasificador procesa un programa Prolog que define fluentes mediante `state_fluent/1` sin que existan ADs en el programa, el índice invertido no contiene entradas relevantes y la inferencia de tipos no detecta ninguna posición estocástica. En consecuencia, todos los fluentes se clasifican como booleanos (base 2). Bajo esta clasificación, el cálculo de strides produce la

serie de potencias binarias $\{1, 2, 4, 8, \ldots\}$, la incorporacion temporal recurre a hechos probabilísticos individuales con probabilidad neutral 0.5, y el espacio de estados se enumera como el producto cartesiano binario. El comportamiento resultante es idéntico al de la arquitectura original.

Esta propiedad permite al usuario adoptar la sintaxis multivaluada de manera incremental, incorporando fluentes multivaluados en modelos existentes sin necesidad de reescribir las partes booleanas del programa.

Con la arquitectura, los algoritmos y la implementación definidos, el sistema cuenta con la capacidad de modelar y resolver MDPs con fluentes multivaluados de manera nativa. El siguiente capítulo somete esta extensión a una evaluación empírica mediante casos de estudio que cuantifican el impacto de la reestructuración tanto en la simplificación del modelado como en el comportamiento computacional del solver.

5. Experimentación y Resultados

El capítulo anterior presentó el diseño y la implementación de la extensión al framework MDP-ProbLog. El presente capítulo somete dicha extensión a una evaluación empírica mediante un conjunto de experimentos diseñados para responder tres preguntas. Primero, si la extensión produce soluciones correctas. Segundo, si simplifica el proceso de modelado. Tercero, cómo afecta al comportamiento computacional del solver. Para ello se utiliza el grid de Mitchell como dominio de referencia, variando el tamaño del grid para analizar las tendencias de escalamiento bajo dos codificaciones del espacio de estados.

5.1. Diseño experimental

5.1.1. Dominio de prueba

El dominio de prueba es una familia de grids basada en el grid world de Mitchell [11], un problema canónico en la literatura de MDPs utilizado para ilustrar el algoritmo de Iteración de Valor. En este dominio, un agente se desplaza por una cuadrícula rectangular de r filas y c columnas. En cada paso de tiempo, el agente selecciona una de cuatro acciones (arriba, abajo, izquierda, derecha). Las transiciones son deterministas: la acción produce el movimiento esperado siempre que la celda destino exista dentro de los límites del grid. Si el agente intenta moverse contra una pared, permanece en su celda actual. El grid contiene una celda meta que otorga una recompensa positiva al ser alcanzada.

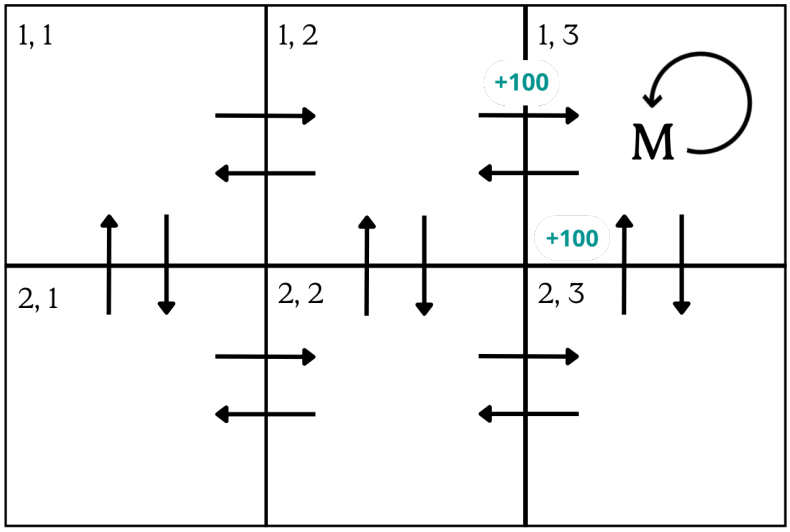

Figura 4: Entorno de navegación en cuadrícula utilizado para la validación de la política óptima. Adaptado de Mitchell [11].

Se seleccionaron cinco tamaños de grid para el análisis de escalamiento. La Tabla 7 resume sus características. La selección sigue tres criterios: incluir el grid original de Mitchell (2×3)

como punto de verificación contra la solución publicada, cubrir un rango de tamaños que permita observar tendencias de escalamiento, e incluir el grid 4×4 como punto de control donde la codificación binaria no produce estados espurios ($16 = 2^4$).

Tabla 7: Tamaños de grid seleccionados para la evaluación experimental.

#	Grid	Celdas	Bits	$\lvert\mathcal{S}\rvert_{\text{bin}}$	$\lvert\mathcal{S}\rvert_{\text{fact}}$	Espurios
1	2×3	6	3	8	6	2
2	3×3	9	4	16	9	7
3	4×4	16	5	32	16	16
4	5×5	25	5	32	25	7
5	6×6	36	6	64	36	28

5.1.2. Codificaciones comparadas

Para cada tamaño de grid se construyeron dos formulaciones del mismo MDP, que difieren exclusivamente en la representación del espacio de estados.

Codificación binaria. La posición del agente se codifica mediante $\lceil \log_2(r \times c) \rceil$ fluentes booleanos. Cada combinación de valores binarios representa un estado del grid, y las reglas de transición se escriben explícitamente para cada par (estado, acción), activando los bits correspondientes al estado destino. Cuando el número de celdas no es una potencia de 2, la codificación produce estados espurios que el modelo debe contemplar mediante reglas adicionales.

```
% 1. Declaracion de fluentes (log2(6) -> 3 bits, genera 8
   combinaciones)
state_fluent(b1).
state_fluent(b2).
state_fluent(b3).

% 2. Manejo de estados espurios (000 y 111 son invalidos en el
   grid 2x3)
blocked :- b1(0), b2(0), b3(0).
blocked :- not(b1(0)), not(b2(0)), not(b3(0)).


% 3. Transiciones representativas
% Movimiento hacia la derecha desde (1,1) [codificado como 1,0,0]
% hacia (1,2) [codificado como 0,1,0]. Se activa b2.
1.0::b2(1) :- b1(0), not(b2(0)), not(b3(0)), right, not(blocked).

% Movimiento hacia abajo desde (1,2) [codificado como 0,1,0]
% hacia (2,2) [codificado como 1,0,1]. Se activan b1 y b3.
1.0::b1(1) :- not(b1(0)), b2(0), not(b3(0)), down.
```

```
1.0::b3(1) :- not(b1(0)), b2(0), not(b3(0)), down.
```

Código 7: Fragmento del modelo binario de control para el grid de 2×3.

Como se observa en el Código 7, la representación binaria desvincula la lógica espacial del problema de su implementación computacional. El motor de inferencia asume la existencia de $2^3 = 8$ estados, obligando al modelador a declarar el predicado `blocked` para excluir las combinaciones `000` y `111` de la evaluación de recompensas y transiciones. Además, transitar hacia un estado que requiere múltiples bits activos exige redactar múltiples cabezas de regla independientes para la misma condición de origen.

Codificación multivaluada factorizada. La posición se codifica mediante dos fluentes multivaluados: `row(i)` con base r y `col(j)` con base c. El espacio de estados resultante tiene exactamente $r \times c$ estados. Las transiciones se expresan en términos de coordenadas. No se requieren reglas de exclusión mutua.

```
% 1. Conocimiento de fondo (Background knowledge)
row(1). row(2).
col(1). col(2). col(3).

% 2. Declaracion de fluentes (Genera exactamente 2x3 = 6 estados
   validos)
state_fluent(x(X), multivalued) :- row(X).
state_fluent(y(Y), multivalued) :- col(Y).

% 3. Transiciones parametrizadas por factor
% Factor Y (columna): El movimiento a la derecha incrementa Y
1.0::y(Y_new, 1) :- y(Y, 0), right, Y_new is Y + 1, col(Y_new),
   not(terminal).
% Choque con la pared: Y se mantiene igual
1.0::y(Y, 1)     :- y(Y, 0), right, Y_new is Y + 1, not(col(Y_new
   )), not(terminal).

% Factor X (fila): El movimiento hacia arriba decrementa X
1.0::x(X_new, 1) :- x(X, 0), up, X_new is X - 1, row(X_new), not(
   terminal).
```

Código 8: Fragmento del modelo multivaluado factorizado para el grid de 2×3.

Como se evidencia en el Código 8, la representación multivaluada factorizada alinea perfectamente la semántica espacial del problema con su implementación lógica. Al definir los fluentes `x(X)` y `y(Y)` explícitamente como `multivalued`, el *framework* construye un espacio *mixed-radix* de base 2×3, iterando exclusivamente sobre los 6 estados semánticamente posibles. Esto elimina por completo la necesidad de codificar reglas de bloqueo (`blocked`) para estados espurios.

Además, el uso de variables lógicas y aritmética (`Y_new is Y + 1`) permite generalizar la dinámica de transición. En lugar de enumerar reglas para cada celda individual como en la codificación binaria, un único par de reglas rige el comportamiento de toda una acción a lo largo del tablero. Esta generalización reduce significativamente la verbosidad del modelo, facilitando su escalabilidad a *grids* de dimensiones arbitrarias sin requerir modificaciones en la lógica subyacente.

5.1.3. Métricas de evaluación

Las métricas se organizan en tres grupos correspondientes a las tres preguntas del experimento.

Las métricas de *corrección funcional* verifican que ambas codificaciones produzcan la misma función de valor óptima $V^*(s)$ y la misma política $\pi^*(s)$, y que estos resultados coincidan con la solución de referencia. La comparación numérica utiliza una tolerancia de 10^{-6}.

Las métricas de *complejidad del modelo* cuantifican el esfuerzo de modelado: número de fluentes declarados, número de cláusulas del programa, número de reglas de exclusión mutua o manejo de estados espurios, y tamaño del espacio de estados $|\mathcal{S}|$.

Las métricas de *impacto computacional* miden el tiempo de preparación del MDP (clasificación, inyección, grounding, compilación), el tiempo de Iteración de Valor, y el tiempo total de resolución.

5.1.4. Protocolo de medición

Las métricas de corrección y complejidad son deterministas y se midieron una sola vez por configuración. Las mediciones de tiempo se ejecutaron en $N = 10$ repeticiones independientes por configuración. Cada repetición consistió en una ejecución completa del pipeline.

5.1.5. Entorno de pruebas

Los experimentos se ejecutaron sobre un equipo con procesador Intel Core Ultra 7 155U (14 núcleos), 16 GiB de memoria RAM y sistema operativo Ubuntu 24.04.4 LTS (kernel Linux 6.17.0, arquitectura x86_64). El intérprete utilizado fue Python 3.12.3 con ProbLog 2.2.10. Se emplearon los backends de compilación d-DNNF y SDD disponibles en ProbLog. Los parámetros del solver se fijaron en $\gamma = 0.9$ (factor de descuento), $\epsilon = 0.01$ (tolerancia de convergencia) y $\epsilon_{\text{thr}} = 10^{-6}$ (umbral de filtrado de ramas con probabilidad despreciable). Estos parámetros se mantuvieron constantes en todas las configuraciones experimentales.

5.2. Verificación de corrección funcional

El primer objetivo de la evaluación es demostrar que la extensión multivaluada produce resultados correctos. Para ello se comparan las soluciones obtenidas por ambas codificaciones entre sí y contra la solución de referencia.

La Tabla 8 presenta los resultados de la verificación cruzada entre ambas codificaciones. Para cada tamaño de grid, se comparan los valores numéricos de $V^*(s)$ obtenidos por la codificación binaria y la codificación factorizada.

Tabla 8: Verificación de corrección funcional. La columna V^* indica si las funciones de valor de ambas codificaciones coinciden numéricamente (tolerancia 10^{-6}). La diferencia máxima observada fue 0.0 en todos los casos.

Grid	V^* **coincide**	**Dif. máxima**	**Iter. binaria**	**Iter. factorizada**
2×3	Sí	0.0	3	3
3×3	Sí	0.0	5	3
4×4	Sí	0.0	7	4
5×5	Sí	0.0	9	5
6×6	Sí	0.0	11	6

Ambas codificaciones producen funciones de valor idénticas para los cinco tamaños evaluados, con una diferencia máxima de 0.0 entre los valores numéricos. Este resultado confirma que la extensión multivaluada preserva la corrección matemática del solver.

Un hallazgo relevante es que el número de iteraciones hasta convergencia difiere entre codificaciones. La codificación binaria requiere consistentemente más iteraciones que la factorizada: de 3 a 11 iteraciones para la binaria, frente a 3 a 6 para la factorizada. Esta diferencia se analiza en la sección de discusión.

En este punto se espera que ambas codificaciones produzcan funciones de valor y políticas idénticas a la solución de referencia para todos los tamaños evaluados, confirmando que la extensión preserva la corrección matemática del solver. El número de iteraciones hasta convergencia debería ser el mismo bajo ambas codificaciones, dado que resuelven el mismo MDP con los mismos parámetros γ y ϵ.

5.3. Simplificación del proceso de modelado

El segundo objetivo evalúa el impacto de la extensión sobre la complejidad del programa ProbLog que el usuario debe escribir. La Tabla 9 presenta las métricas estructurales de ambas codificaciones para cada tamaño de grid.

Tabla 9: Complejidad estructural de los programas ProbLog. La codificación factorizada mantiene un tamaño constante de 54 líneas y 27 cláusulas independientemente del tamaño del grid, mientras que la codificación binaria crece de 92 a 421 líneas.

	Fluentes		**Reglas esp./excl.**		**Cláusulas**		**Líneas totales**	
Grid	Bin.	Fact.	Bin.	Fact.	Bin.	Fact.	Bin.	Fact.
2×3	3	2	15	0	19	27	92	54
3×3	4	2	30	0	55	27	133	54
4×4	2	2	49	0	61	27	165	54
5×5	5	2	103	0	196	27	260	54
6×6	6	2	266	0	335	27	421	54

Los datos revelan tres observaciones. La primera es que la codificación factorizada utiliza exactamente 2 fluentes (fila y columna) y produce un programa de 54 líneas y 27 cláusulas en todos los tamaños evaluados. La estructura del programa es independiente del tamaño del grid porque las reglas de transición se expresan en términos de coordenadas, no de combinaciones de bits.

La segunda observación es que la codificación binaria requiere un número creciente de reglas para manejar los estados y codificar las transiciones. El número de reglas de exclusión o manejo de estados espurios crece de 15 en el grid 2×3 a 266 en el grid 6×6. Este crecimiento refleja el esfuerzo de modelado que la codificación factorizada elimina por completo.

La tercera observación concierne al tamaño del espacio de estados. La Tabla 10 presenta la comparación directa.

Tabla 10: Tamaño del espacio de estados y factor de inflación. El factor de inflación mide cuántas veces más grande es el espacio binario respecto al factorizado.

Grid	**Celdas**	$\lvert\mathcal{S}\rvert_{\text{bin}}$	$\lvert\mathcal{S}\rvert_{\text{fact}}$	**Factor de inflación**
2×3	6	8	6	1.33×
3×3	9	16	9	1.78×
4×4	16	16	16	1×
5×5	25	32	25	1.28×
6×6	36	64	36	1.78×

El factor de inflación varía entre 1.28× y 1.78× dependiendo de la relación entre el número de celdas y la potencia de 2 más cercana. El grid 4×4 presenta el factor mas bajo (1×) debido a

que en este caso el tamaño del espacio de estados es una potencia de 2. Este resultado indica que la codificación binaria implementada no optimiza el número de bits al mínimo teórico.

5.4. Impacto computacional

El tercer objetivo cuantifica el efecto de la representación sobre el comportamiento computacional del solver. Se reportan el tamaño del circuito compilado y los tiempos de las fases del pipeline.

5.4.1. Tamaño del circuito compilado

El tamaño del circuito compilado refleja la complejidad de la fórmula proposicional que ProbLog debe construir para evaluar las probabilidades de transición. Dado que la codificación binaria introduce estados espurios que el compilador debe representar (incluso para descartarlos), se espera que los circuitos de la codificación binaria sean de mayor tamaño. El grid 4×4 es el punto de control de esta hipótesis: al no tener estados espurios, cualquier diferencia en el tamaño del circuito para ese grid se debe exclusivamente a la estructura de las reglas de transición, no a la presencia de combinaciones inválidas.

5.4.2. Tiempos de ejecución

La Tabla 11 presenta los tiempos de las fases del pipeline para cada configuración.

Tabla 11: Tiempos de ejecución por fase del pipeline (en segundos). Se reporta media ± desviación estándar sobre 10 repeticiones.

Grid	**Cod.**	t_{prepare} (s)	t_{VI} (s)	t_{total} (s)
2×3	Bin.	0.023 ± 0.004	0.014 ± 0.002	0.037 ± 0.005
	Fact.	0.041 ± 0.006	0.021 ± 0.004	0.063 ± 0.007
3×3	Bin.	0.049 ± 0.004	0.035 ± 0.001	0.084 ± 0.005
	Fact.	0.048 ± 0.005	0.035 ± 0.006	0.083 ± 0.008
4×4	Bin.	0.117 ± 0.005	0.128 ± 0.003	0.245 ± 0.006
	Fact.	0.063 ± 0.005	0.069 ± 0.001	0.132 ± 0.005
5×5	Bin.	0.210 ± 0.007	0.211 ± 0.006	0.421 ± 0.011
	Fact.	0.072 ± 0.003	0.130 ± 0.003	0.202 ± 0.004
6×6	Bin.	0.407 ± 0.024	0.724 ± 0.035	1.131 ± 0.047
	Fact.	0.096 ± 0.006	0.247 ± 0.006	0.343 ± 0.010

En los grids pequeños (2×3 y 3×3), los tiempos de ambas codificaciones son comparables y del orden de decenas de milisegundos. A partir del grid 4×4, la codificación factorizada comienza a mostrar tiempos menores en ambas fases del pipeline. La diferencia se amplifica con el tamaño del grid: en el grid 6×6, la codificación binaria requiere 1.131 segundos frente a 0.343 segundos de la factorizada.

La Tabla 12 cuantifica la relación de tiempos entre ambas codificaciones.

Tabla 12: Relación de tiempos entre codificaciones (*speedup*). Un valor mayor que 1.0 indica que la codificación factorizada es más rápida. Se calcula como $t_{\text{bin}}/t_{\text{fact}}$ usando las medias.

Grid	**Speedup preparación**	**Speedup VI**	**Speedup total**
2×3	0.57×	0.63×	0.59×
3×3	1.03×	1.00×	1.02×
4×4	1.87×	1.84×	1.85×
5×5	2.91×	1.62×	2.09×
6×6	4.25×	2.93×	3.30×

5.5. Discusión

Los resultados experimentales permiten evaluar la extensión propuesta desde los tres ejes planteados en el diseño experimental.

5.5.1. Corrección y convergencia

Ambas codificaciones producen funciones de valor idénticas para los cinco tamaños evaluados, confirmando que la extensión preserva la corrección del solver. Sin embargo, la codificación binaria requiere consistentemente más iteraciones hasta convergencia (de 3 a 11) que la factorizada (de 3 a 6). Esta diferencia se explica por el mayor tamaño del espacio de estados en la codificación binaria. Los estados espurios reciben inicialmente un valor de cero y el algoritmo de Iteración de Valor necesita más iteraciones para propagar los valores desde la celda meta hasta los estados más lejanos a través de un espacio más grande. El efecto se amplifica con el tamaño del grid: en el grid 6×6, la codificación binaria necesita casi el doble de iteraciones que la factorizada.

5.5.2. Simplificación del modelado

La codificación factorizada produce un programa de tamaño constante (54 líneas, 27 cláusulas) independientemente del tamaño del grid, mientras que la codificación binaria crece de 92 a 421 líneas. El factor que impulsa este crecimiento es el número de reglas de manejo de estados espurios, que pasa de 15 a 266 en el rango evaluado. La codificación factorizada elimina estas reglas por completo, ya que la exclusión mutua se garantiza por la semántica de las Disyunciones Anotadas.

Este resultado tiene una consecuencia práctica directa. Al escalar el dominio, la codificación binaria requiere que el modelador escriba un número creciente de reglas proporcional al tamaño del grid, mientras que la codificación factorizada no requiere cambios en la estructura del programa. El único ajuste necesario es la actualización de las Disyunciones Anotadas que definen los valores de fila y columna.

5.5.3. Comportamiento computacional

Los tiempos de ejecución revelan un patrón que no era evidente a priori: la codificación factorizada es más lenta que la binaria para el grid más pequeño (2×3, speedup de 0.59×), comparable para el grid 3×3 (speedup de 1.02×), y progresivamente más rápida a partir del grid 4×4 hasta alcanzar un speedup de 3.30× en el grid 6×6.

La inversión de rendimiento en el grid 2×3 se explica por el costo fijo de las Disyunciones Anotadas. Para grids pequeños, el overhead de construir las ADs, inyectarlas en la ClauseDB y compilar los nodos `choice` resultantes supera el ahorro obtenido por la reducción del espacio de estados. A medida que el grid crece, la reducción del espacio de estados y del número de cláusulas compensa este costo fijo, y la ventaja de la codificación factorizada se amplifica.

La fase de preparación del MDP (clasificación, inyección, grounding y compilación) es donde la diferencia es más pronunciada. En el grid 6×6, la preparación de la codificación binaria toma 0.407 segundos frente a 0.096 de la factorizada, un factor de 4.25×. Esto es consistente con el hecho de que el programa binario contiene 335 cláusulas frente a 27 de la factorizada: más cláusulas implican un grounding más costoso y un circuito compilado de mayor tamaño.

5.5.4. Relación con las predicciones del Marco Teórico

En la Sección **??** se argumentó que las Disyunciones Anotadas tienen el potencial de producir circuitos más compactos al evitar la representación de combinaciones imposibles. Los datos de tiempo de preparación son consistentes con esta predicción: la fase que incluye la compilación del circuito es la que muestra la mayor diferencia entre codificaciones, lo que sugiere que el circuito generado por la codificación factorizada es efectivamente más pequeño. Una verificación directa del tamaño del circuito (número de nodos SDD o d-DNNF) confirmaría esta observación y se identifica como trabajo futuro.

5.5.5. Limitaciones de la evaluación

Los experimentos se realizaron sobre dominios de grid con transiciones deterministas y un único tipo de recompensa (celda meta). Dominios con transiciones estocásticas, múltiples fuentes de recompensa, o fluentes de estado adicionales (más allá de la posición) podrían exhibir comportamientos distintos. Laa codificación binaria utilizada en estos experimentos optimiza el número de bits al mínimo teórico $\lceil \log_2(k) \rceil$ en todos los casos, como se observó en el grid 4×4 donde se emplean 4 bits para 16 celdas.

Los resultados presentados en este capítulo proporcionan la evidencia empírica para evaluar el cumplimiento de los objetivos planteados en la Introducción. El siguiente capítulo sintetiza las contribuciones de este trabajo y plantea las líneas de investigación futura.

Índice de figuras

1. Representación por coordenadas vs. codificación booleana con estados inalcanzables para el grid 2×3. 3
2. Grid de navegación 2x3 estocástico. Se incorporan transiciones probabilísticas. 14
3. Política óptima: (a) una flecha indica la acción óptima $\pi^*(r, c)$; (b) muestra el valor $V^*(r, c)$ calculado por Iteración de Valor. 23
4. Entorno de navegación en cuadrícula utilizado para la validación de la política óptima. Adaptado de Mitchell [11]. 38

Índice de cuadros

1. Codificación binaria de las 6 celdas de un grid de 2×3. 3
2. Escalamiento de estados espurios en la codificación binaria por tamaño de grid. 4
3. Codificación binaria forzada para una variable ternaria en el dominio *Factory*. 5
4. Comparativa de los enfoques del estado del arte. 12
5. Mundos posibles inducidos por la semántica de distribución sobre el Código 1. 18
6. Distribución generada por la AD del Código 3. Cada selección corresponde a un único destino posible. 20
7. Tamaños de grid seleccionados para la evaluación experimental. 39
8. Verificación de corrección funcional. La columna V^* indica si las funciones de valor de ambas codificaciones coinciden numéricamente (tolerancia 10^{-6}). La diferencia máxima observada fue 0.0 en todos los casos. 42
9. Complejidad estructural de los programas ProbLog. La codificación factorizada mantiene un tamaño constante de 54 líneas y 27 cláusulas independientemente del tamaño del grid, mientras que la codificación binaria crece de 92 a 421 líneas. 43
10. Tamaño del espacio de estados y factor de inflación. El factor de inflación mide cuántas veces más grande es el espacio binario respecto al factorizado. 43
11. Tiempos de ejecución por fase del pipeline (en segundos). Se reporta media $\pm$ desviación estándar sobre 10 repeticiones. 44
12. Relación de tiempos entre codificaciones (*speedup*). Un valor mayor que 1.0 indica que la codificación factorizada es más rápida. Se calcula como $t_{\text{bin}}/t_{\text{fact}}$ usando las medias. 45

Índice de algoritmos

1. Inferencia de tipo por intersección de orígenes estocásticos 30
2. Cálculo Expected Value 35

Referencias

[1] Luc De Raedt, Angelika Kimmig y Hannu Toivonen. "ProbLog: A probabilistic prolog and its application in link discovery". En: *IJCAI*. Vol. 7. 2007, págs. 2462-2467.

[2] Taisuke Sato. "A statistical learning method for logic programs with distribution semantics". En: *Proceedings of the 12th International Conference on Logic Programming (ICLP'95)*. Citeseer, 1995.

[3] Daan Fierens et al. "Inference and learning in probabilistic logic programs using weighted Boolean formulas". En: *Theory and Practice of Logic Programming* 15.3 (2015), págs. 358-401.

[4] Richard Bellman. *Dynamic Programming*. Princeton University Press, 1957.

[5] Martin L Puterman. *Markov decision processes: discrete stochastic dynamic programming*. John Wiley & Sons, 2014.

[6] Thiago P Bueno et al. "Markov decision processes specified by probabilistic logic programming: Representation and solution". En: *2016 5th Brazilian Conference on Intelligent Systems (BRACIS)*. IEEE. 2016, págs. 337-342.

[7] Craig Boutilier, Thomas Dean y Steve Hanks. "Decision-theoretic planning: Structural assumptions and computational leverage". En: *Journal of Artificial Intelligence Research* 11 (1999), págs. 1-94.

[8] Jesse Hoey et al. "SPUDD: Stochastic Planning using Decision Diagrams". En: *Proceedings of the 15th Conference on Uncertainty in Artificial Intelligence (UAI-99)*. 1999, págs. 279-288.

[9] Davide Nitti, Vaishak Belle y Luc De Raedt. "Planning in discrete and continuous Markov decision processes by probabilistic programming". En: *Machine Learning* 104.2-3 (2016), págs. 327-342.

[10] Héctor Avilés et al. "Autonomous Behavior Selection For Self-driving Cars Using Probabilistic Logic Factored Markov Decision Processes". En: *Applied Artificial Intelligence* 38.1 (2024), pág. 2304942.

[11] Tom M. Mitchell. *Machine Learning*. New York: McGraw-Hill, 1997.

[12] Richard S Sutton y Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 2018.

[13] Joost Vennekens, Sofie Verbaeten y Maurice Bruynooghe. "Logic programs with annotated disjunctions". En: *International Conference on Logic Programming*. Springer, 2004, págs. 431-445.

[14] Craig Boutilier, Richard Dearden y Moisés Goldszmidt. "Stochastic dynamic programming with factored representations". En: *Artificial Intelligence* 121.1-2 (2000), págs. 49-107.

[15] Scott Sanner. "Relational Dynamic Influence Diagram Language (RDDL): Language Description". Unpublished Manuscript, Australian National University. 2010.

[16] Luc De Raedt y Angelika Kimmig. "Probabilistic (logic) programming concepts". En: *Machine Learning* 100.1 (2015), págs. 5-47.

[17] Guy Van den Broeck et al. "DTProbLog: A Decision-Theoretic Probabilistic Prolog". En: *Proceedings of the Twenty-Fourth AAAI Conference on Artificial Intelligence (AAAI-10)*. AAAI Press, 2010, págs. 1217-1222.

[18] Joost Vennekens, Marc Denecker y Maurice Bruynooghe. "CP-logic: A language of causal probabilistic events and its relation to logic programming". En: *Theory and Practice of Logic Programming* 9.3 (2009), págs. 245-308.

[19] David Poole. "The Independent Choice Logic for Modelling Multiple Agents under Uncertainty". En: *Artificial Intelligence* 94.1-2 (1997), págs. 7-56.

[20] Adnan Darwiche y Pierre Marquis. "A knowledge compilation map". En: *Journal of Artificial Intelligence Research* 17 (2002), págs. 229-264.

[21] Adnan Darwiche. "SDD: A New Canonical Representation of Propositional Knowledge Bases". En: *Proceedings of the 22nd International Joint Conference on Artificial Intelligence (IJCAI)*. 2011, págs. 819-826.

[22] Adnan Darwiche. "A differential approach to inference in Bayesian networks". En: *Journal of the ACM* 50.3 (2003), págs. 280-305.

[23] Anton Dries et al. "ProbLog2: Probabilistic Logic Programming". En: *Machine Learning and Knowledge Discovery in Databases*. Ed. por Albert Bifet et al. Cham: Springer International Publishing, 2015, págs. 312-315.

[24] Stuart Jonathan Russell y Peter Norvig. *Artificial Intelligence: A Modern Approach*. English. Third. England: Pearson Education, 2016. ISBN: 9781292153964.